Practical No.1

Aim:- Introduction to Excel

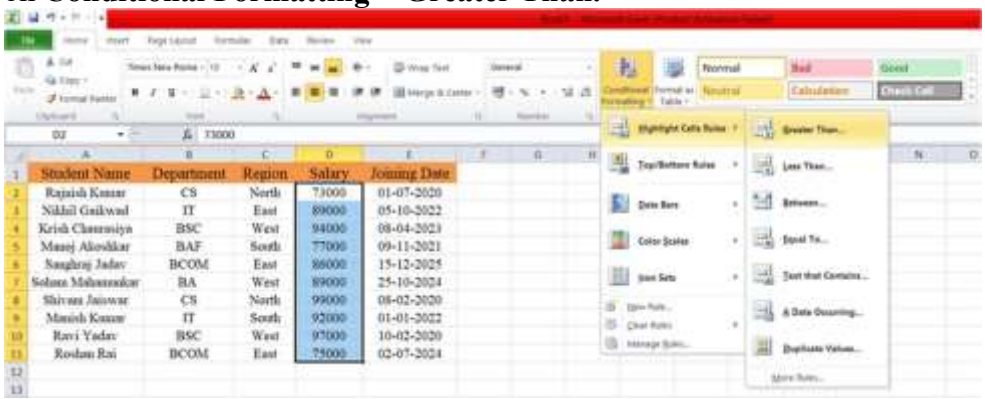
1. Perform conditional formatting on a dataset using various criteria.
2. Create a pivot table to analyze and summarize data.
3. Use VLOOKUP function to retrieve information from a different worksheet or table.
4. Perform what-if analysis using Goal Seek to determine input values for desired output.

Steps :-

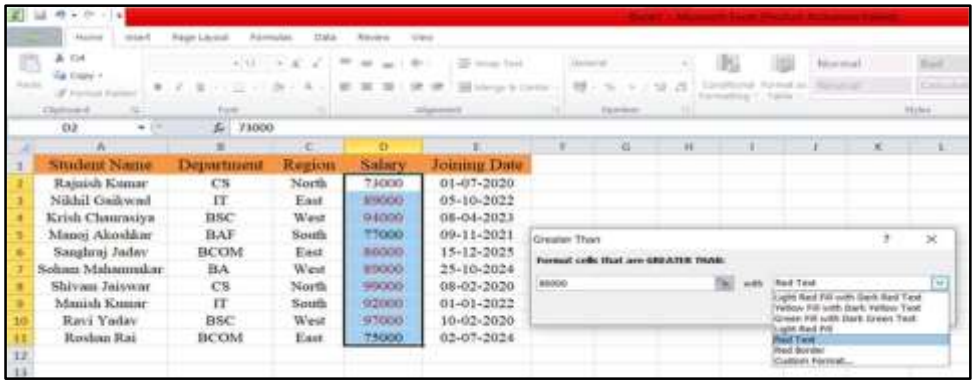
1. Perform conditional formatting on a dataset using various criteria.

Apply conditional formatting to the **salary** column to highlight cells where the salary is Greater than **$80000**

Step 1: Select the column on which you want to apply conditional formatting. Then on home screen click on **Conditional Formatting > Greater Than.**

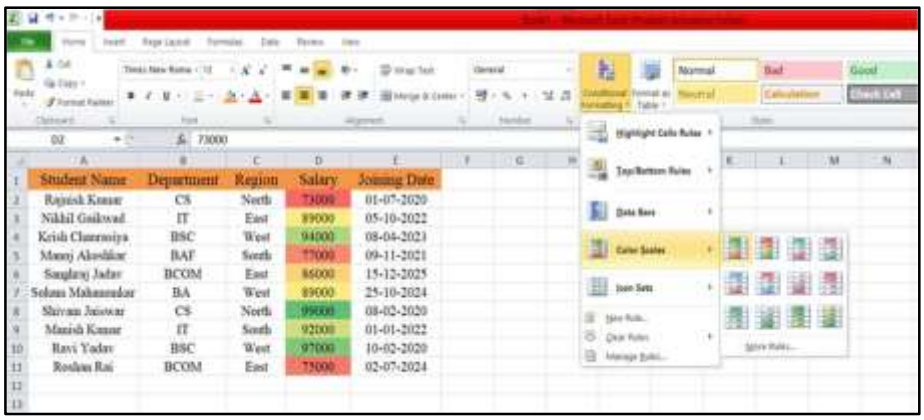


Step 2: Now, In input field give the value as "80000" and in combobox select "Red Text" and click on OK.



Condition formatting is applied.
Usle a color scale to visually show the distribution of salary values
(e.g. green for high and red for low salary)

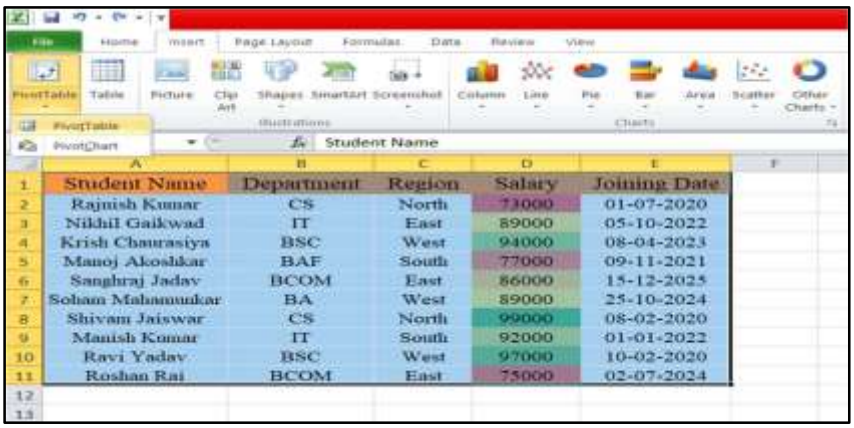
Step: Click on conditional formatting > Color Scales there select 1st one



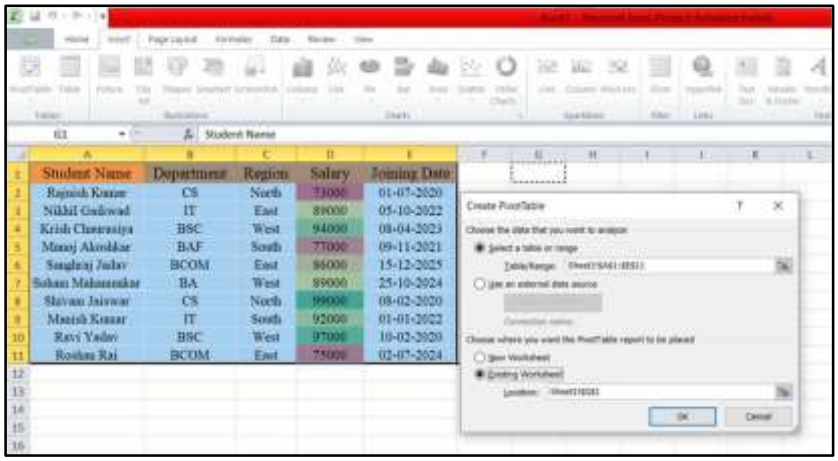
2. Create a pivot table to analyze and summarize data.

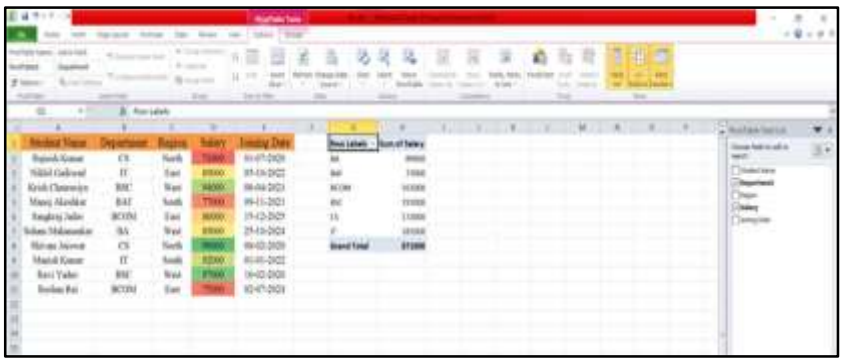
Create a pivot table that summarize the total salary by department

Step 1: Select the entire table then click on Insert > Pivot Table



<u>Step 2:</u> Here, select "Existing Worksheet" then click on any cell in background there the pivot table will be created. then click on OK

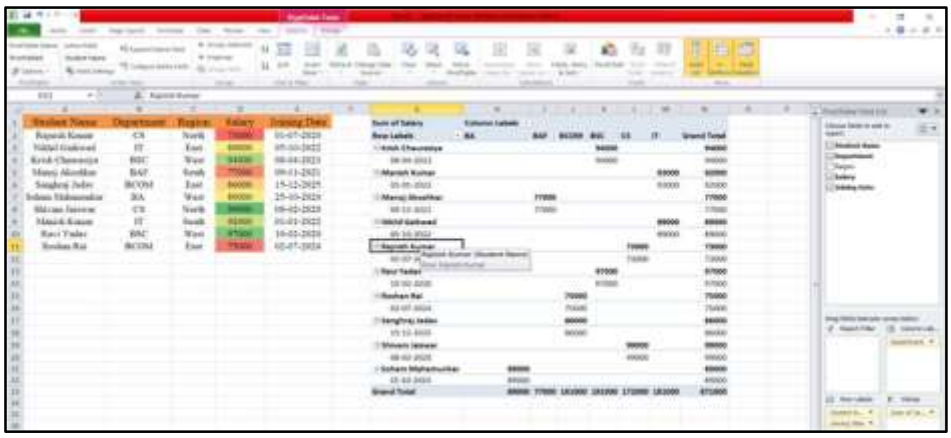




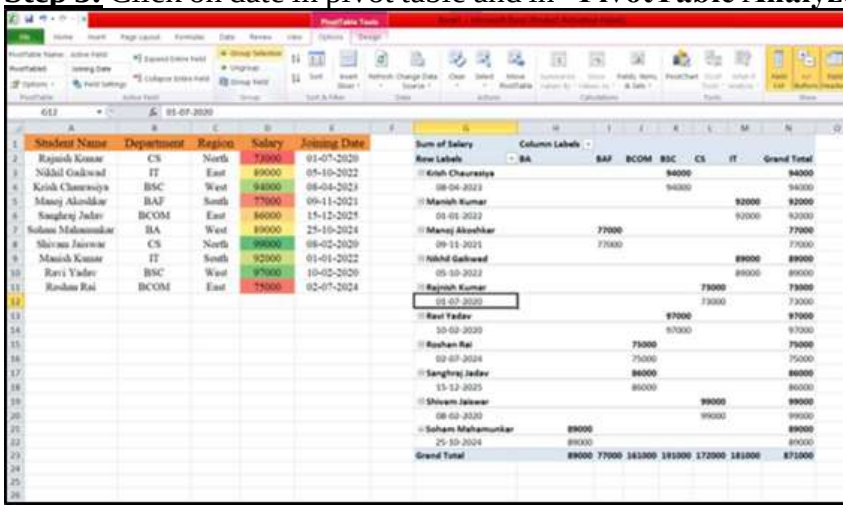
Add a filter to the pivot table to allow the analysis of employee data for specific years (based on the Joining Date).

Step 1: To do this first select the **"Joining Date"** column right-click then click on **"Format cells"** there choose in category choose **"Date".**

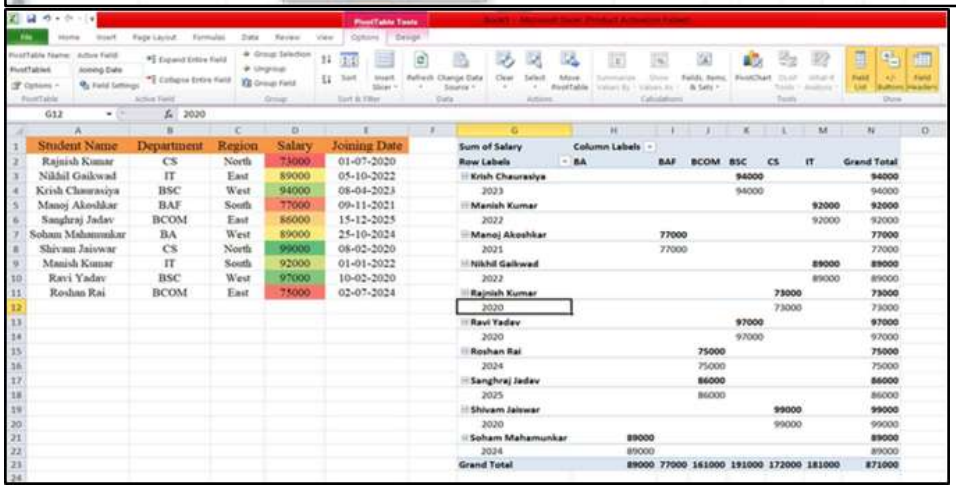
Step 2: Create the pivot table like this



Step 3: Click on date in pivot table and in **"PivotTable Analyze"** click on **"Group Selection"**



In that select **"Years"** click on OK



3. Use VLOOKUP function to retrieve information from a different worksheet or table. Imagine you have a list of products and their prices, and you want to find the price of a specific product

	A	B	C	D	E
1	Sr no	Product Id	Product Name	Price	
2	1	101	Apple	50	
3	2	102	Orange	100	
4	3	103	Banana	40	
5	4	104	Kiwi	150	
6	5	105	Sweet Lime	100	
7	6	106	Carrot	25	
8	7	107	Dragon	80	
9	8	108	Custard Apple	160	
10					

Step 1: Identify the Search Value (Lookup Value) **We want the price of** "KIWI"**.** Type KIWI in any cell.

Example: Cell F5 = KIWI

Step 2: Select the Table Range (Where Excel Will Search) The table range must include: The column where Excel will search (Product

Name)
The column from which the result will come (Price)
In this case:

C2:D9

C2 = first product
name D9 = last
price

Step 3: Identify the Column Number

VLOOKUP counts the columns inside the selected range, not in the whole sheet. In the range C2:D9:
C = column 1

D = column 2

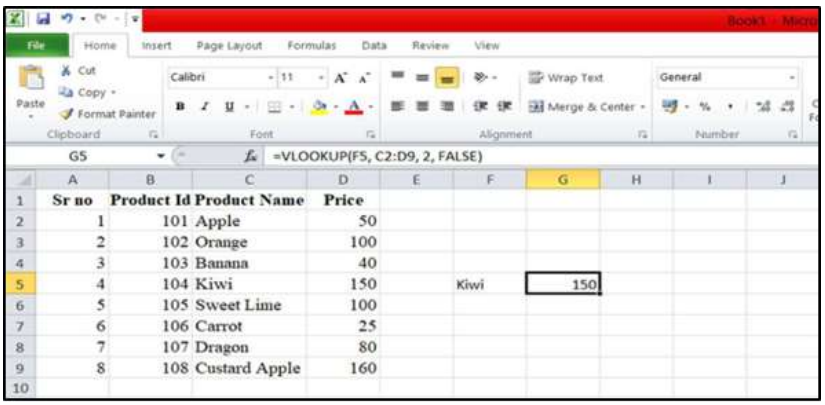
We want the value from the 2nd column (Price column). So column number = 2

Step 4: Choose Exact Match or Approximate Match We want the exact value for "KIWI".
So we use: FALSE

Step 5: Write the VLOOKUP Formula Now combine all four parts:
=VLOOKUP(F5, C2:D9, 2, FALSE)

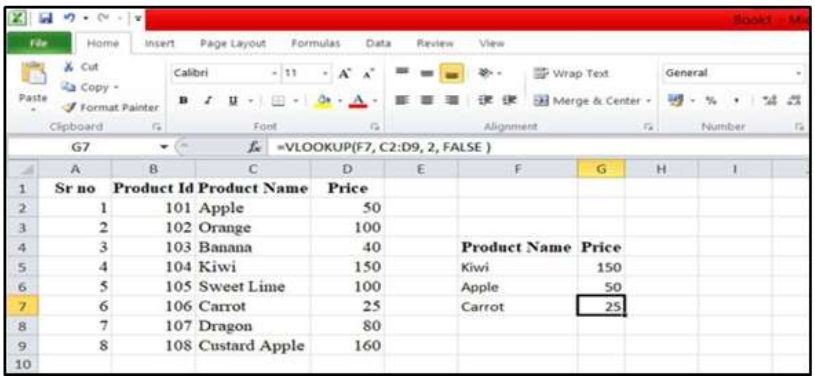
Where:

F5 = KIWI (lookup value) C2:D9 = table range
2 = price column
FALSE = exact match



G5 =VLOOKUP(F5, C2:D9, 2, FALSE)

	A	B	C	D	E	F	G	H	I	J
1	Sr no	Product Id	Product Name	Price						
2	1	101	Apple	50						
3	2	102	Orange	100						
4	3	103	Banana	40						
5	4	104	Kiwi	150		Kiwi	150			
6	5	105	Sweet Lime	100						
7	6	106	Carrot	25						
8	7	107	Dragon	80						
9	8	108	Custard Apple	160						
10										

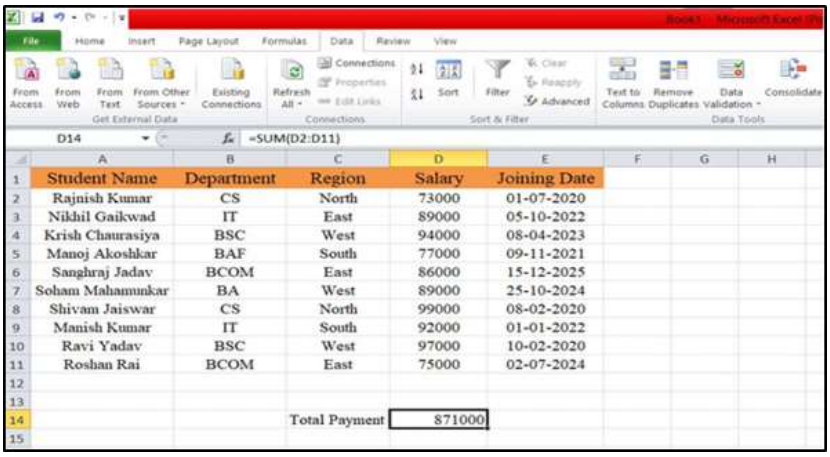


G7 =VLOOKUP(F7, C2:D9, 2, FALSE)

	A	B	C	D	E	F	G	H	I
1	Sr no	Product Id	Product Name	Price					
2	1	101	Apple	50					
3	2	102	Orange	100					
4	3	103	Banana	40		Product Name	Price		
5	4	104	Kiwi	150		Kiwi	150		
6	5	105	Sweet Lime	100		Apple	50		
7	6	106	Carrot	25		Carrot	25		
8	7	107	Dragon	80					
9	8	108	Custard Apple	160					
10									

Goal Seek

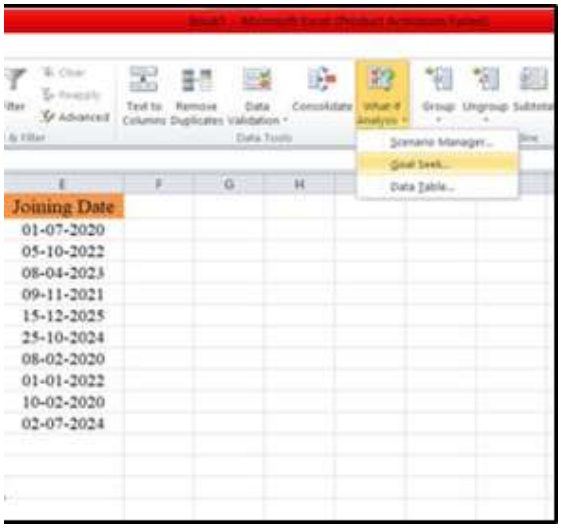
Open the Excel worksheet.



D14 =SUM(D2:D11)

	A	B	C	D	E	F	G	H
1	Student Name	Department	Region	Salary	Joining Date			
2	Rajnish Kumar	CS	North	73000	01-07-2020			
3	Nikhil Gaikwad	IT	East	89000	05-10-2022			
4	Krish Chaurasiya	BSC	West	94000	08-04-2023			
5	Manoj Akoshkar	BAF	South	77000	09-11-2021			
6	Sanghraj Jadav	BCOM	East	86000	15-12-2025			
7	Soham Mahamunkar	BA	West	89000	25-10-2024			
8	Shivam Jaiswar	CS	North	99000	08-02-2020			
9	Manish Kumar	IT	South	92000	01-01-2022			
10	Ravi Yadav	BSC	West	97000	10-02-2020			
11	Roshan Rai	BCOM	East	75000	02-07-2024			
12								
13								
14			Total Payment	871000				
15								

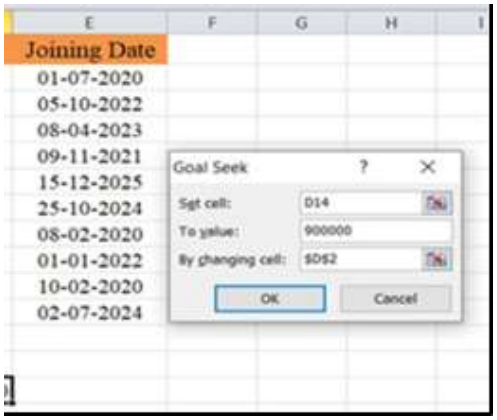
Click on Data tab and then select What-If Analysis. From the What-If Analysis dropdown menu, click on Goal Seek.



The Goal Seek dialog box appears.

Set Cell: Select the cell containing the formula. To Value: Enter the desired result value.

By Changing Cell: Select the input cell to be



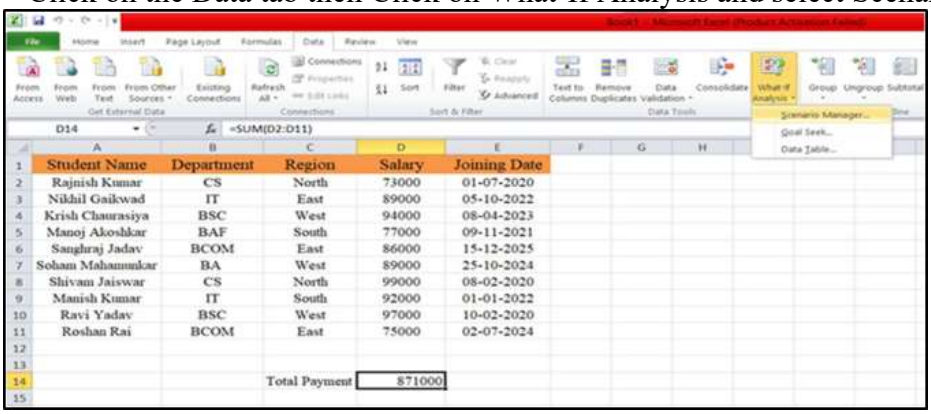
Click on OK. Excel calculates the required input value to achieve the desired output.

The Goal Seek result is displayed successfully.

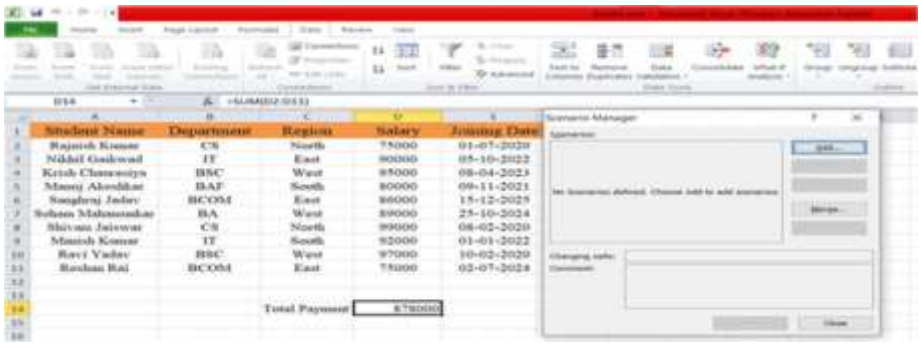
Scenario Manager

Open the Excel worksheet and identify the cells whose values you want to change for different scenarios.

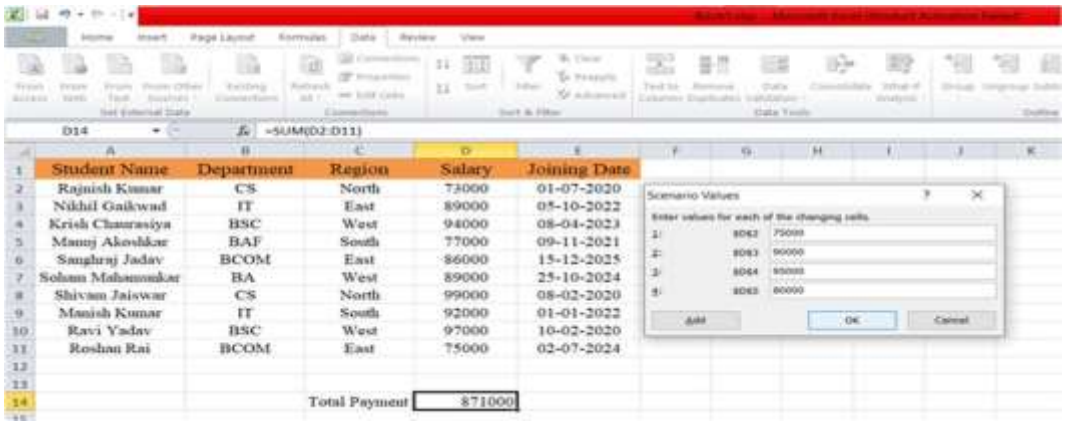
Click on the Data tab then Click on What-If Analysis and select Scenario Manager



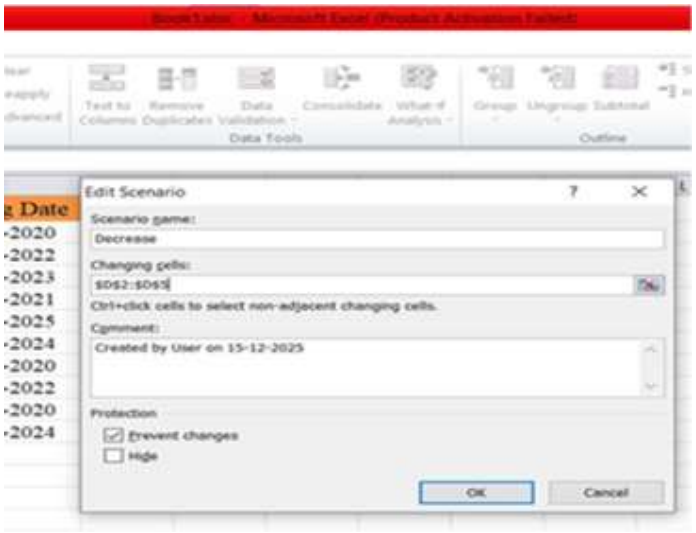
The Scenario Manager dialog box appears. Click on Add to create a new scenario



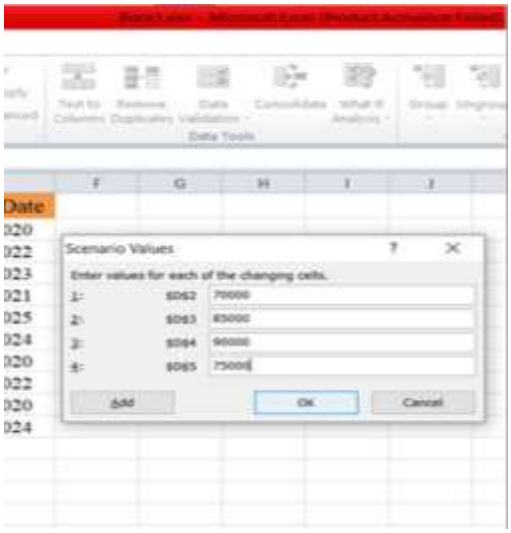
Enter the Scenario Name for e.g., Increase, and select the Changing Cells, then click on OK. Enter the increased values in the changing cells and click on OK



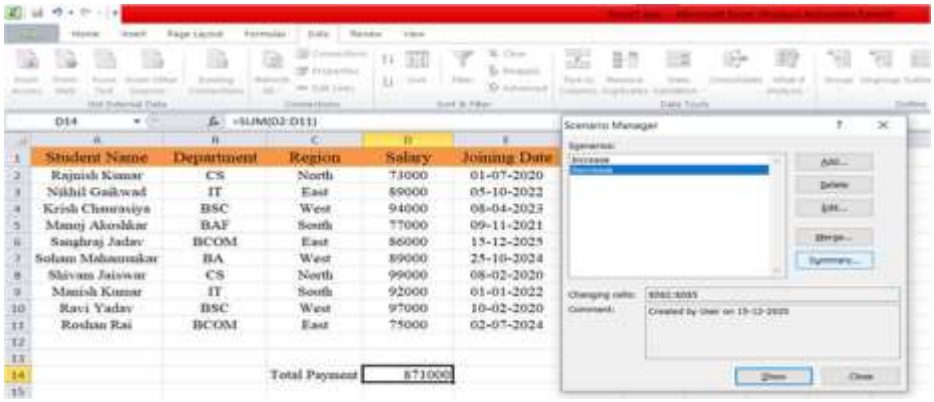
Again, click on Add in Scenario Manager, Enter the Scenario Name as Decrease. Select the same Changing Cells and click on OK.



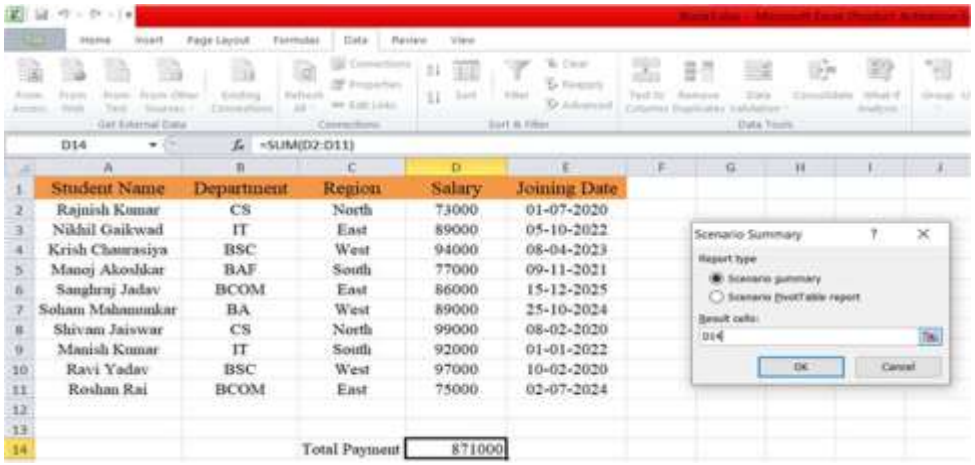
Enter the decreased values in the changing cells and click on OK.



Select the created scenario and click on Show.

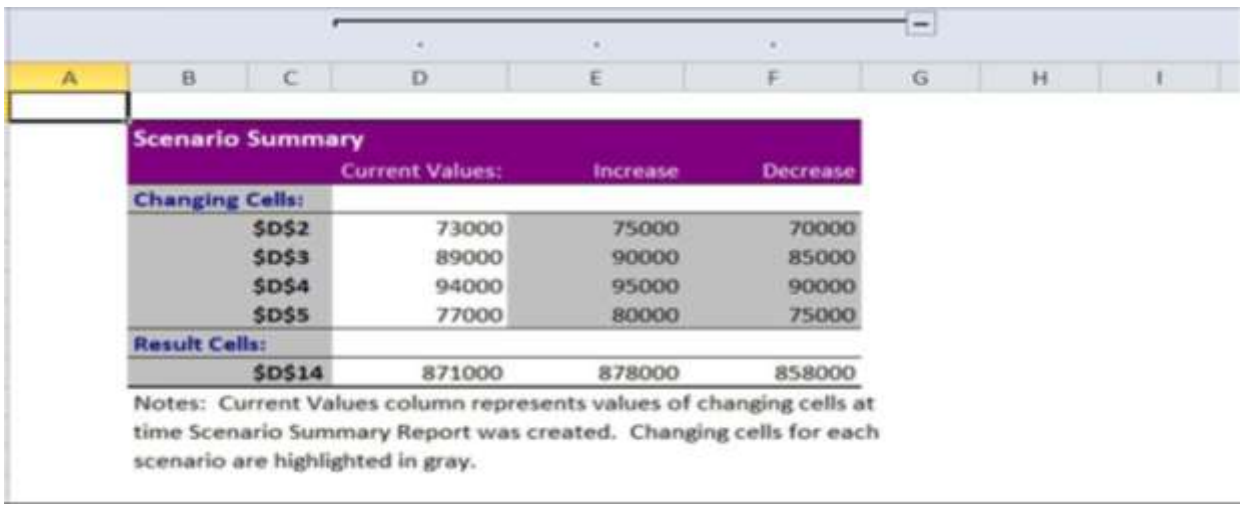


Select the created scenario summary click on ok to view the result.



Student Name	Department	Region	Salary	Joining Date
Rajnish Kumar	CS	North	73000	01-07-2020
Nikhil Gaikwad	IT	East	89000	05-10-2022
Krish Chaurasiya	BSC	West	94000	08-04-2023
Manoj Akoshkar	BAF	South	77000	09-11-2021
Sanghraj Jadav	BCOM	East	86000	15-12-2025
Soham Mahamunkar	BA	West	89000	25-10-2024
Shivam Jaiswar	CS	North	99000	08-02-2020
Manish Kumar	IT	South	92000	01-01-2022
Ravi Yadav	BSC	West	97000	10-02-2020
Roshan Rai	BCOM	East	75000	02-07-2024
		Total Payment	871000	

OUTPUT :-



Scenario Summary	Current Values:	Increase	Decrease
Changing Cells:			
D2	73000	75000	70000
D3	89000	90000	85000
D4	94000	95000	90000
D5	77000	80000	75000
Result Cells:			
D14	871000	878000	858000

Notes: Current Values column represents values of changing cells at time Scenario Summary Report was created. Changing cells for each scenario are highlighted in gray.

Practical No.2

Aim:-Data Frames and Basic Data Pre-processing:-

- **Read data from CSV and JSON files into a data frame.**
- **Perform basic data pre-processing tasks such as handling missing values an outliers .**
- **Manipulate and transform data using functions like filtering, sorting, and grouping.**

Step1:- open jupyter Notebook and upload the playTENNIES.csv and sample_data.json file in jupyter Notebook.

Input:

```
import pandas as pd
csv_file="PlayTennis.csv";
df_csv=pd.read_csv(csv_file)
print( df_csv.tail())
print("shape:")
print(df_csv.shape)
print("columns:")
print(df_csv.columns)
print("dtypes:")
print(df_csv.dtypes)
print(df_csv)
```

Output :

```
     Outlook Temperature Humidity    Wind Play Tennis
9       Rain        Mild   Normal    Weak         Yes
10     Sunny        Mild   Normal  Strong         Yes
11  Overcast        Mild     High  Strong         Yes
12  Overcast         Hot   Normal    Weak         Yes
13      Rain        Mild     High  Strong          No
shape:
(14, 5)
columns:
Index(['Outlook', 'Temperature', 'Humidity', 'Wind', 'Play Tennis'], dtype='object')
dtypes:
Outlook        object
Temperature    object
Humidity       object
Wind           object
Play Tennis    object
dtype: object
```

```
     Outlook Temperature Humidity    Wind Play Tennis
0      Sunny         Hot     High    Weak          No
1      Sunny         Hot     High  Strong          No
2   Overcast         Hot     High    Weak         Yes
3       Rain        Mild     High    Weak         Yes
4       Rain        Cool   Normal    Weak         Yes
5       Rain        Cool   Normal  Strong          No
6   Overcast        Cool   Normal  Strong         Yes
7      Sunny        Mild     High    Weak          No
8      Sunny        Cool   Normal    Weak         Yes
9       Rain        Mild   Normal    Weak         Yes
10     Sunny        Mild   Normal  Strong         Yes
11  Overcast        Mild     High  Strong         Yes
12  Overcast         Hot   Normal    Weak         Yes
13      Rain        Mild     High  Strong          No
```

Input :

```
json_file="Sample_data.json";
df_json=pd.read_json(json_file)
print("\njson_data:",df_json.head())
print("\n head(2):",df_json.head(2))
print("\n tail(2):",df_json.tail(2))
```

Output :

```
json_data:    id    name department  salary joining_date
0   1   Alice         HR   50000   2020-01-15
1   2     Bob        IIT   51000   2020-01-16
2   3   David        IIT   52000   2020-01-17
3   4  Marcus  Marketing   53000   2020-01-18
4   5  Conner         HR   54000   2020-01-19

 head(2):    id   name department  salary joining_date
0   1  Alice         HR   50000   2020-01-15
1   2    Bob        IIT   51000   2020-01-16

 tail(2):    id    name department  salary joining_date
3   4  Marcus  Marketing   53000   2020-01-18
4   5  Conner         HR   54000   2020-01-19
```

Input:

```
import pandas as pd
csv_file="PlayTennis.csv";
df_csv=pd.read_csv(csv_file)
print(df_csv)
```

Output:

```
     Outlook Temperature Humidity    Wind Play Tennis  Number of people
0      Sunny         Hot     High    Weak          No                 1
1      Sunny         NaN     High  Strong          No               100
2   Overcast         Hot     High    Weak         Yes                 3
3       Rain        Mild     High    Weak         Yes                 4
4       Rain        Cool   Normal    Weak         Yes                 2
5       Rain        Cool   Normal  Strong          No                 1
6   Overcast        Cool   Normal  Strong         Yes                 3
7      Sunny        Mild     High    Weak          No                 2
8      Sunny        Cool   Normal    Weak         Yes                 4
9       Rain        Mild   Normal    Weak         Yes                 5
10     Sunny        Mild   Normal  Strong         Yes                 2
11  Overcast        Mild     High  Strong         Yes                 4
12  Overcast         Hot   Normal    Weak         Yes                 3
13      Rain        Mild     High  Strong          No                 2
```

Input:

```
print("\n missing values in data set")
print(df_csv.isnull().sum())
```

Output:

```
 missing values in data set
Outlook            0
Temperature        1
Humidity           0
Wind               0
Play Tennis        0
Number of people   0
dtype: int64
```

Input:

```
df_csv["Temperature"].fillna(df_csv["Temperature"].mode()[0],inplace=True)
print("\n Dataset after filling missing values with modes:")
print(df_csv)
```

```
 Dataset after filling missing values with mins:
     Outlook Temperature Humidity    Wind Play Tennis  Number of people
0      Sunny         Hot     High    Weak          No                 1
1      Sunny        Mild     High  Strong          No               100
2   Overcast         Hot     High    Weak         Yes                 3
3       Rain        Mild     High    Weak         Yes                 4
4       Rain        Cool   Normal    Weak         Yes                 2
5       Rain        Cool   Normal  Strong          No                 1
6   Overcast        Cool   Normal  Strong         Yes                 3
7      Sunny        Mild     High    Weak          No                 2
8      Sunny        Cool   Normal    Weak         Yes                 4
9       Rain        Mild   Normal    Weak         Yes                 5
10     Sunny        Mild   Normal  Strong         Yes                 2
11  Overcast        Mild     High  Strong         Yes                 4
12  Overcast         Hot   Normal    Weak         Yes                 3
13      Rain        Mild     High  Strong          No                 2
```

Input:

```
df_csv["Number of people"].fillna(df_csv["Number of people"].mean(),inplace=True)
print("\n Dataset after filling missing values with mins:")
```

```
print(df_csv)
```

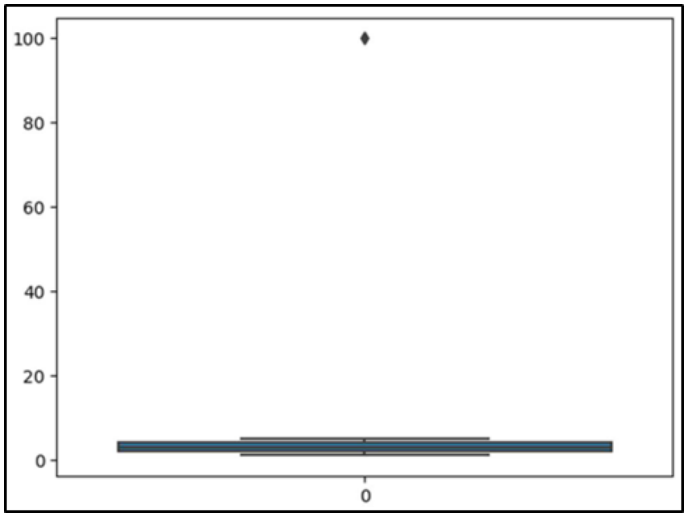
Output:

```
Dataset after filling missing values with mins:
     Outlook Temperature Humidity    Wind Play Tennis  Number of people
0      Sunny         Hot     High    Weak          No                 1
1      Sunny        Mild     High  Strong          No               100
2   Overcast         Hot     High    Weak         Yes                 3
3       Rain        Mild     High    Weak         Yes                 4
4       Rain        Cool   Normal    Weak         Yes                 2
5       Rain        Cool   Normal  Strong          No                 1
6   Overcast        Cool   Normal  Strong         Yes                 3
7      Sunny        Mild     High    Weak          No                 2
8      Sunny        Cool   Normal    Weak         Yes                 4
9       Rain        Mild   Normal    Weak         Yes                 5
10     Sunny        Mild   Normal  Strong         Yes                 2
11  Overcast        Mild     High  Strong         Yes                 4
12  Overcast         Hot   Normal    Weak         Yes                 3
13      Rain        Mild     High  Strong          No                 2
```

Input:

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.boxplot(data=df_csv["Number of people"])
plt.show()
```

Output:



Input :

```
Q1=df_csv["Number of people"].quantile(0.25)
Q2=df_csv["Number of people"].quantile(0.75)
IQR=Q2-Q1
lower_bound=Q1-3+IQR
upper_bound=Q2+3+IQR
extreme_outliners=df_csv[(df_csv["Number of people"]<lower_bound)|
(df_csv["Number of people"]>upper_bound)]
print(extreme_outliners)
df_cleaned=df_csv[(df_csv["Number of people"]>=lower_bound)& (df_csv["Number of
people"]<=upper_bound)] print("\n",df_cleaned)
```

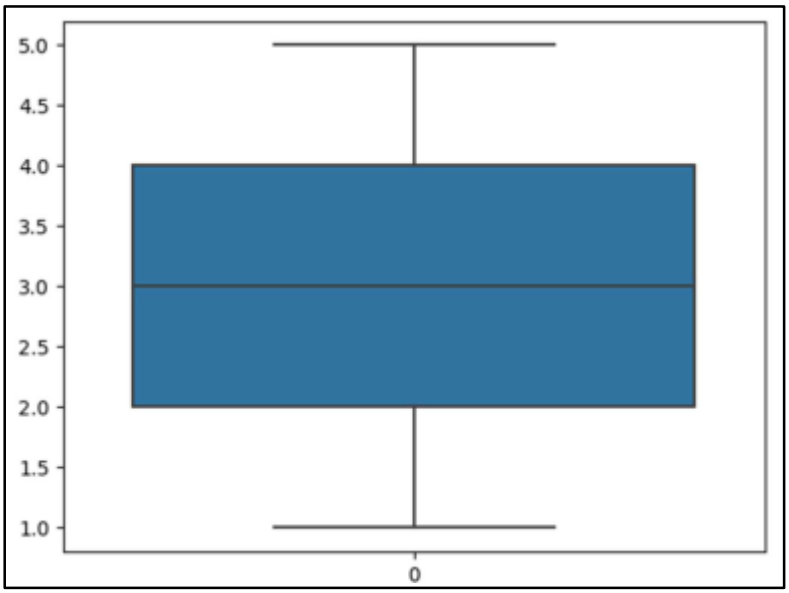
Input:

	Outlook	Temperature	Humidity	Wind	Play Tennis	Number of people
1	Sunny	NaN	High	Strong	No	100

	Outlook	Temperature	Humidity	Wind	Play Tennis	Number of people
0	Sunny	Hot	High	Weak	No	1
2	Overcast	Hot	High	Weak	Yes	3
3	Rain	Mild	High	Weak	Yes	4
4	Rain	Cool	Normal	Weak	Yes	2
5	Rain	Cool	Normal	Strong	No	1
6	Overcast	Cool	Normal	Strong	Yes	3
7	Sunny	Mild	High	Weak	No	2
8	Sunny	Cool	Normal	Weak	Yes	4
9	Rain	Mild	Normal	Weak	Yes	5
10	Sunny	Mild	Normal	Strong	Yes	2
11	Overcast	Mild	High	Strong	Yes	4
12	Overcast	Hot	Normal	Weak	Yes	3
13	Rain	Mild	High	Strong	No	2

```
import seaborn as sns
sns.boxplot(data=df_cleaned["Number of people"])
```

Output:



Practical No.3

Code:

Write input one by one

Step 1: import pandas as pd

Step 2: from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder

Step 3: from sklearn.compose import ColumnTransformer

Step 4: from sklearn.pipeline import Pipeline

Step 5:

```
#define the data
data = {
   'product': [
      'Apple_juice', 'Banana_Smoothie', 'Orange_jam', 'Grape_Jelly',
      'Kivi_parfait', 'Mango_Chutney', 'Pinapple_Sorbet',
      'Strawberry_yogurt', 'Blueberry_pie', 'Cherry_Salad'
   ],
   'Category': [
      'Apple', 'Banana', 'Orange', 'Grapes', 'Kivi',
      'Mango', 'Pinapple', 'Strawberry', 'Blueberry', 'Cherry'
   ],
   'grade': ['2', '1', '1', '3', '2', '1', '1', '2', '1', '2'],
   'Cost': [500, 200, 100, 600, 400, 500, 800, 200, 700, 900],
   'Sales': [1200, 2000, 3000, 4000, 5000, 8000, 9000, 7000, 3000, 5000]
}
```

Step 6:

```
#create a data frame
df=pd.DataFrame(data)
print("orignal data")
print(df)
```

Output :

```
orignal data
              product    Category grade  Cost  Sales
0         Apple_juice       Apple     2   500   1200
1     Banana_Smoothie      Banana     1   200   2000
2          Orange_jam      Orange     1   100   3000
3         Grape_Jelly      Grapes     3   600   4000
4        Kivi_parfait        Kivi     2   400   5000
5       Mango_Chutney       Mango     1   500   8000
6     Pinapple_Sorbet    Pinapple     1   800   9000
7   Strawberry_yogurt  Strawberry     2   200   7000
8       Blueberry_pie   Blueberry     1   700   3000
9        Cherry_Salad      Cherry     2   900   5000
```

Step 7:

```
#Step1 :-Feature Scalling(Standarization/Normalization)
n_columns=['grade','Cost','Sales']
s_std=StandardScaler()
s_nor=MinMaxScaler()
df_s_std=pd.DataFrame(s_std.fit_transform(df[n_columns]),columns=n_colum
ns)
df_s_nor=pd.DataFrame(s_nor.fit_transform(df[n_columns]),columns=n_colum
ns)
df_scaled=pd.concat([df_s_std,df.drop(n_columns,axis=1)],axis=1)
print("Data set after feature Scaling")
print(df_scaled)
```

Output :

```
Data set after feature Scaling
      grade      Cost     Sales            product    Category
0  0.603023  0.039253 -1.429242        Apple_juice       Apple
1 -0.904534 -1.138350 -1.104414    Banana_Smoothie      Banana
2 -0.904534 -1.530884 -0.698380         Orange_jam      Orange
3  2.110579  0.431788 -0.292345        Grape_Jelly      Grapes
4  0.603023 -0.353281  0.113690       Kivi_parfait        Kivi
5 -0.904534  0.039253  1.331794      Mango_Chutney       Mango
6 -0.904534  1.216856  1.737828    Pinapple_Sorbet    Pinapple
7  0.603023 -1.138350  0.925759  Strawberry_yogurt  Strawberry
8 -0.904534  0.824322 -0.698380      Blueberry_pie   Blueberry
9  0.603023  1.609391  0.113690       Cherry_Salad      Cherry
```

Step 8:

```
#Step2:-feature dummification
#Identify categorical columns
categorical_columns=["product","Category"]
#Create a column transformer for dummification
preprocessor=ColumnTransformer(
transformers=[
('categorical',OneHotEncoder(),categorical_columns)],
remainder='passthrough'
)
```

```
#apply the column transformer to the dataset
df_dummified=pd.DataFrame(preprocessor.fit_transform(df).toarray())
#Get column names after dummification
column_names=preprocessor.get_feature_names_out()
#assign column names to the new dataframe
df_dummified.columns=column_names
#Display the dataset after feature dummification
print("\n Dataset after feature dummification")
print(df_dummified)
```

Output :

```
 Dataset after feature dummification
   categorical__product_Apple_juice  categorical__product_Banana_Smoothie  \
0                               1.0                                   0.0
1                               0.0                                   1.0
2                               0.0                                   0.0
3                               0.0                                   0.0
4                               0.0                                   0.0
5                               0.0                                   0.0
6                               0.0                                   0.0
7                               0.0                                   0.0
8                               0.0                                   0.0
9                               0.0                                   0.0

   categorical__product_Blueberry_pie  categorical__product_Cherry_Salad  \
0                                 0.0                                0.0
1                                 0.0                                0.0
2                                 0.0                                0.0
3                                 0.0                                0.0
4                                 0.0                                0.0
5                                 0.0                                0.0
6                                 0.0                                0.0
7                                 0.0                                0.0
8                                 1.0                                0.0
9                                 0.0                                1.0

   categorical__product_Grape_Jelly  categorical__product_Kivi_parfait  \
0                               0.0                                0.0
1                               0.0                                0.0
2                               0.0                                0.0
3                               1.0                                0.0
4                               0.0                                1.0
5                               0.0                                0.0
6                               0.0                                0.0
7                               0.0                                0.0
8                               0.0                                0.0
9                               0.0                                0.0

   categorical__product_Mango_Chutney  categorical__product_Orange_jam  \
0                                 0.0                              0.0
1                                 0.0                              0.0
2                                 0.0                              1.0
3                                 0.0                              0.0
4                                 0.0                              0.0
5                                 1.0                              0.0
6                                 0.0                              0.0
7                                 0.0                              0.0
8                                 0.0                              0.0
9                                 0.0                              0.0
```

```
   categorical__product_Pinapple_Sorbet  \
0                                   0.0
1                                   0.0
2                                   0.0
3                                   0.0
4                                   0.0
5                                   0.0
6                                   1.0
7                                   0.0
8                                   0.0
9                                   0.0

   categorical__Category_Grapes  categorical__Category_Kivi  \
0                           0.0                         0.0
1                           0.0                         0.0
2                           0.0                         0.0
3                           1.0                         0.0
4                           0.0                         1.0
5                           0.0                         0.0
6                           0.0                         0.0
7                           0.0                         0.0
8                           0.0                         0.0
9                           0.0                         0.0

   categorical__Category_Mango  categorical__Category_Orange  \
0                          0.0                          0.0
1                          0.0                          0.0
2                          0.0                          1.0
3                          0.0                          0.0
4                          0.0                          0.0
5                          1.0                          0.0
6                          0.0                          0.0
7                          0.0                          0.0
8                          0.0                          0.0
9                          0.0                          0.0

   categorical__Category_Pinapple  categorical__Category_Strawberry  \
0                             0.0                               0.0
1                             0.0                               0.0
2                             0.0                               0.0
3                             0.0                               0.0
4                             0.0                               0.0
5                             0.0                               0.0
6                             1.0                               0.0
7                             0.0                               1.0
8                             0.0                               0.0
9                             0.0                               0.0

   remainder__grade  remainder__Cost  remainder__Sales
0               2.0            500.0            1200.0
1               1.0            200.0            2000.0
2               1.0            100.0            3000.0
3               3.0            600.0            4000.0
4               2.0            400.0            5000.0
5               1.0            500.0            8000.0
6               1.0            800.0            9000.0
7               2.0            200.0            7000.0
8               1.0            700.0            3000.0
9               2.0            900.0            5000.0

[10 rows x 23 columns]
```

Practical No.4

❖ **Hypothesis Testing**

1. **Formulate null and alternative hypotheses for a given problem.**
2. **Conduct a hypothesis test using appropriate statistical tests (e.g., t-test, chi- square test).**
3. **Interpret the results and draw conclusions based on the test outcomes.**

Code :

Program 1:

```
import scipy.stats as stats
g1=[72,88,64,74,67,79,85,75,89,77]
assumed_mean=70
t_stat, p_val = stats.ttest_1samp(g1,assumed_mean)
print("T-Value:", t_stat)
print("P-Value:", p_val)
alpha = 0.05
if(p_val<alpha):
   print(f"Reject the null hypothesis: The mean test score is significantly
different from {assumed_mean}")
else:
  print(f"Fail to reject the null hypothesis: There is no significant difference
between the sample mean and {assumed_mean}")
```

Output :

```
T-Value: 2.6249999999999996
P-Value: 0.027583862030934297
Reject the null hypothesis: The mean test score is significantly different from 70
```

Program 2:

```
import scipy.stats as stats
import numpy as np
g1=[72,88,64,74,67,79,85,75,89,77]
assumed_mean=70
t_stat,p_val=stats.ttest_1samp(g1,assumed_mean)
alpha=0.05
df = len(g1) -1
critical_t=stats.t.ppf(1 -alpha,df)
print("T-value:", t_stat)
print("P-value:", p_val)
print("Critical t-value:", critical_t)
if(np.abs(t_stat) > critical_t):
```

```
  print("Reject the null hypothesis: There is a significant difference in between
the0 two groups.")
else:
  print("Fail to reject the null hypothesis: There is no significant difference in
between the two groups.")
```

Output:

```
T-value: 2.6249999999999996
P-value: 0.027583862030934297
Critical t-value: 1.8331129326536335
Reject the null hypothesis: There is a significant difference in between the0 two groups.
```

Program 3:

```
import scipy.stats as stats
import numpy as np
g1 = [85,95,100,80,90,97,104,95,88,92,94,99]
g2 = [83,85,96,92,100,104,94,95,88,90,93,94]
t_stat, p_val = stats.ttest_ind(g1, g2)
alpha = 0.05
df = (len(g1) - 1) + (len(g2) - 1)
critical_t = stats.t.ppf(1 - alpha/2, df)
print("T-value", t_stat)
print("P-value", p_val)
print("Critical t-value", critical_t)
if(np.abs(t_stat) > critical_t):
  print("Reject the null hypothesis: There is a significant difference in between
the two groups.")
else:
  print("Fail to reject the null hypothesis: There is no significant difference in
between the two groups.")
```

Output:

```
T-value 0.16119885358127883
P-value 0.8734079490313532
Critical t-value 2.0738730679040147
Fail to reject the null hypothesis: There is no significant difference in between the two groups.
```

Program 4:

```
import scipy.stats as stats
import numpy as np
g1 = [85,68,67,84,98,60,94,80,94,98,95,80,85,87,75]
g2 = [70,90,80,89,88,86,78,87,90,86,92,94,99,93,86]
t_stat, p_val = stats.ttest_rel(g1, g2)
alpha = 0.05
df = len(g2) - 1
critical_t = stats.t.ppf(1 - alpha/2, df)
print("T-value", t_stat)
```

```
print("P-value", p_val)
print("Critical t-value", critical_t)
if(np.abs(t_stat)>critical_t):
  print("Reject the null hypothesis: There is a significant difference in between the two groups.")
else:
  print("Fail to reject the null hypothesis: There is no significant difference in between the two groups.")
```

Output:

```
T-value -1.125690756981246
P-value 0.27922443475463854
Critical t-value 2.1447866879169273
Fail to reject the null hypothesis: There is no significant difference in between the two groups.
```

Program 5:

```
from scipy.stats import chi2_contingency
contingency_table = [[50,75],[125,175],[90,30],[45,10]]
chi2_stat, p_val, dof, expected_table = chi2_contingency(contingency_table)
alpha = 0.05
print("Chi-square statistic:", chi2_stat)
print("P-value", p_val)
print("Degrees of freedom:", dof)
print("Expected Values: \n", expected_table)
if(p_val>alpha):
  print("Reject the null hypothesis: There is an association between the 2 groups.")
else:
  print("Fail to reject the null hypothesis: The 2 groups have no association")
```

Output:

```
Chi-square statistic: 65.0116290828193
P-value 4.987377462630171e-14
Degrees of freedom: 3
Expected Values:
 [[ 64.58333333  60.41666667]
 [155.         145.        ]
 [ 62.          58.        ]
 [ 28.41666667  26.58333333]]
Fail to reject the null hypothesis: The 2 groups have no association
```

Practical No.5

ANOVA (Analysis of Variance)

1. **Perform one-way ANOVA to compare means across multiple groups.**
2. **Conduct post-hoc tests to identify significant differences between group means.**

Step 1: Load the CSV File

```
dietData <-
read.csv(file="C:\\Users\\spdc\\Desktop\\DataScience\\90_Data_File.csv",
header = TRUE, sep = ",")
print(dietData)
```

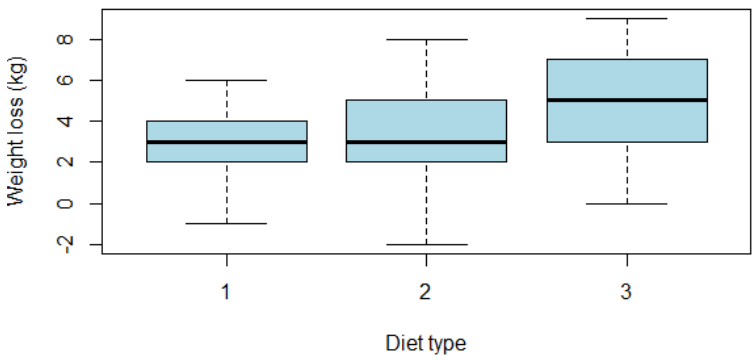
Output:

```
   Diet weight.week.1 weight.week.8 weight.Loss
1     1            59            55           4
2     1            61            55           6
3     1            65            64           1
4     1            65            62           3
5     1            66            63           3
6     1            67            65           2
7     1            68            66           2
8     1            70            65           5
9     1            71            69           2
10    1            71            68           3
11    1            73            72           1
12    1            73            70           3
13    1            73            69           4
14    1            83            82           1
15    1            72            73          -1
16    1            73            72           1
17    1            75            71           4
18    1            79            75           4
19    1            81            76           5
20    1            81            79           2
21    1            84            80           4
22    1            86            83           3
23    1            88            83           5
24    1            89            86           3
25    2            61            61           0
26    2            96            88           8
27    2            59            61          -2
28    2            59            57           2
29    2            60            58           2
30    2            62            58           4
31    2            63            56           7
32    2            64            63           1
33    2            64            61           3
34    2            64            60           4
35    2            66            63           3
```

Step 2: Boxplot for Data Visualization

```
boxplot(dietData$Weight.Loss ~ dietData$Diet, data = dietData,
col="light blue", ylab = "Weight loss (kg)", xlab = "Diet type")
```

Output:



Step 3: Convert Diet to a Factor

```
dietData$Diet <- as.factor(dietData$Diet)
```

Step 4: Perform One-Way ANOVA

table <- aov(dietData$Weight.Loss ~ dietData$Diet, data = dietData)
summary(table)

Output:

```
              Df Sum Sq Mean Sq F value  Pr(>F)
dietData$Diet  2   69.4    34.7   6.551 0.00239 **
Residuals     75  397.3     5.3
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Step 5: Tukey's Post-Hoc Test

TukeyHSD(table)

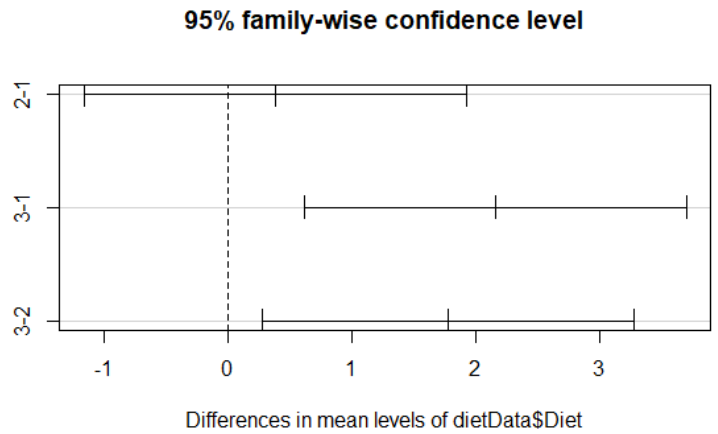
Output:

```
  Tukey multiple comparisons of means
    95% family-wise confidence level

Fit: aov(formula = dietData$Weight.Loss ~ dietData$Diet, data = dietData)

$`dietData$Diet`
         diff        lwr      upr     p adj
2-1 0.3796296 -1.1643251 1.923584 0.8270005
3-1 2.1574074  0.6134526 3.701362 0.0036876
3-2 1.7777778  0.2799216 3.275634 0.0159151
```

Step 6: Visualize Tukey Test Results

plot(TukeyHSD(table))

Output:



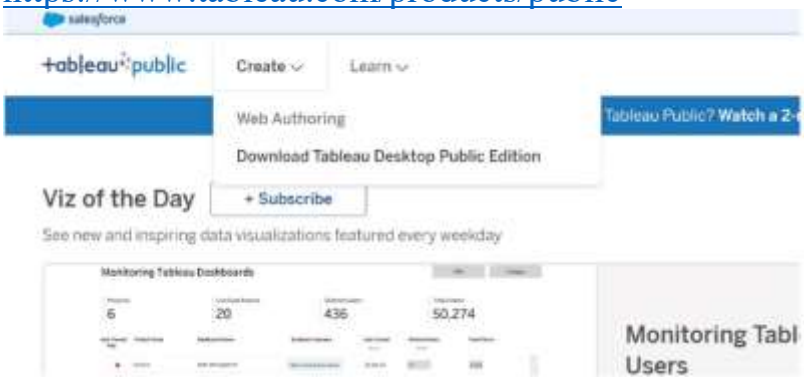
Practical No.6

Aim:-Data Visualization and Storytelling

- **Create meaningful visualizations using data visualization tools**
- **Combine multiple visualizations to tell a compelling data story.**
- **Present the findings and insights in a clear and concise manner.**

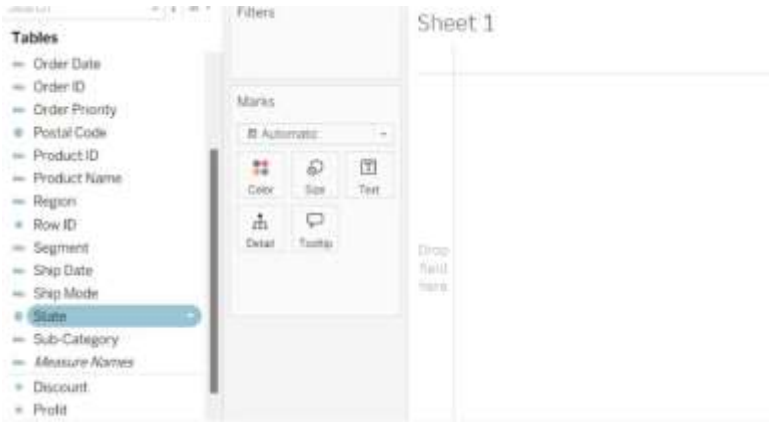
Steps:-

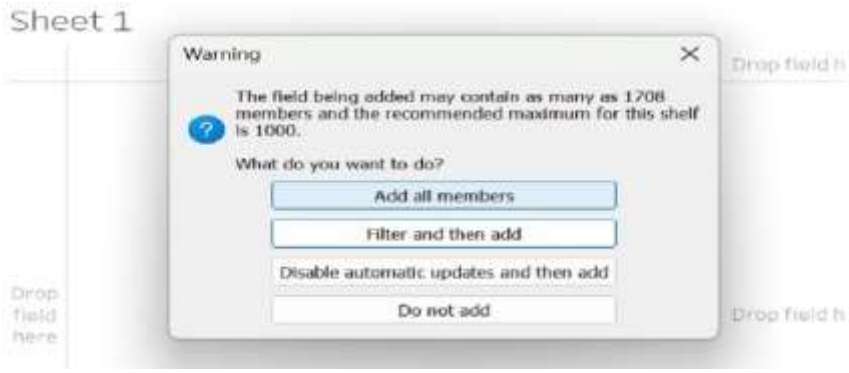
Step1:- Download the tableau from the given link :-
https://www.tableau.com/products/public



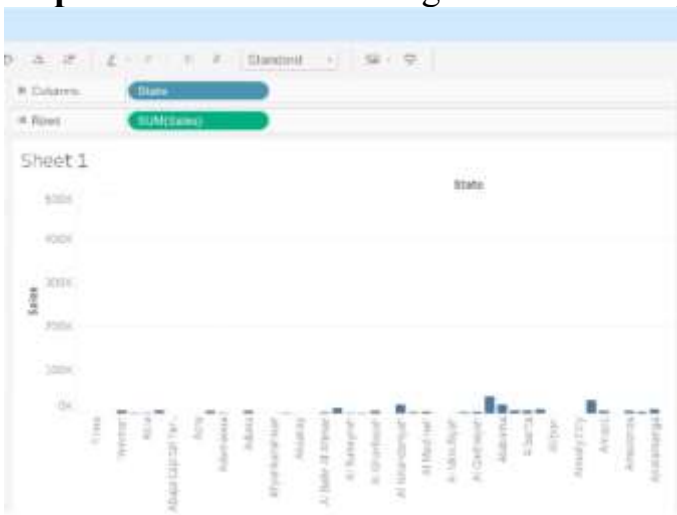
Step2 :- Open a tableau and To a File section select Microsoft XL and add the dataset file.

Step3:- Click on sheet1 and In table section drag a State and drop on column section and click on add all members.

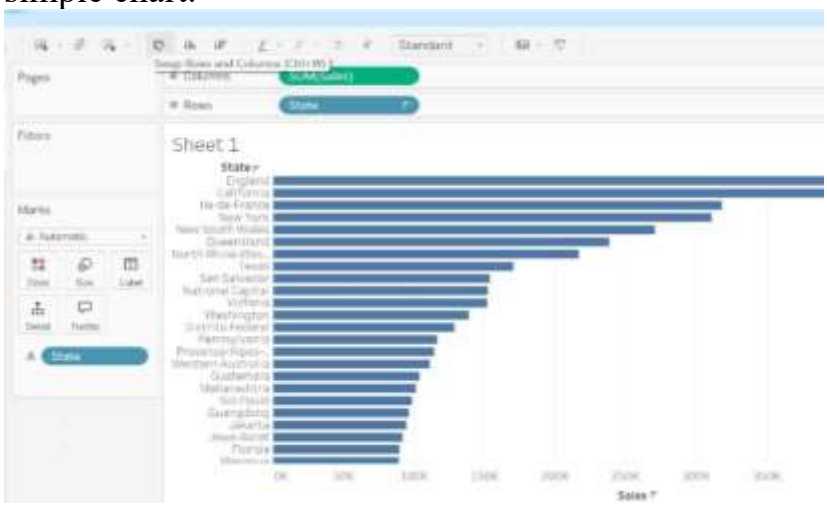




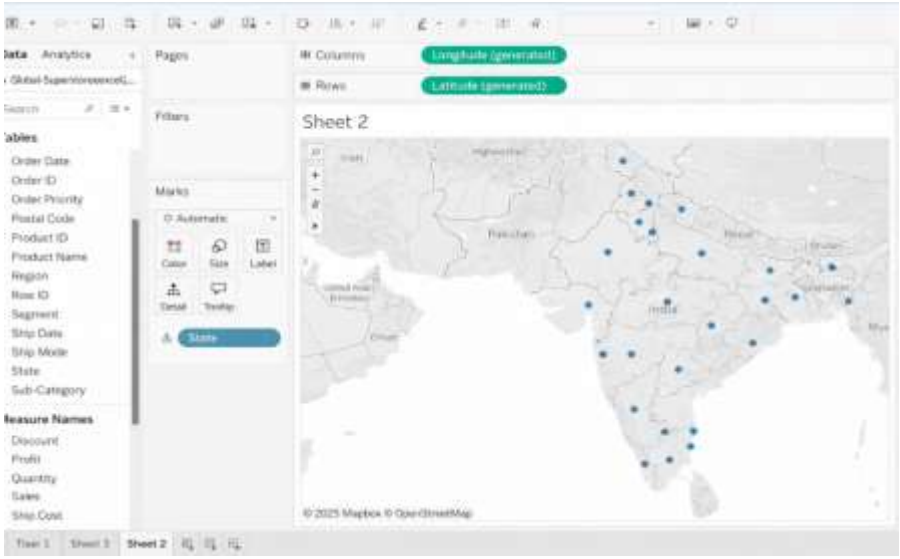
Step4 :- In table section drag a Sales and drop on Row section



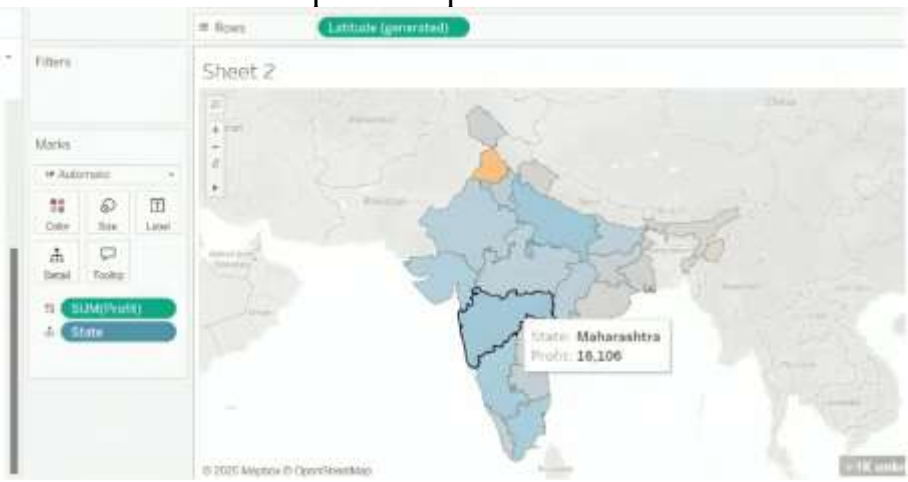
Step5:-If you want to swap the rows and arrange in descending order then you click on icon in header you see the simple representation and save this file with simple chart.



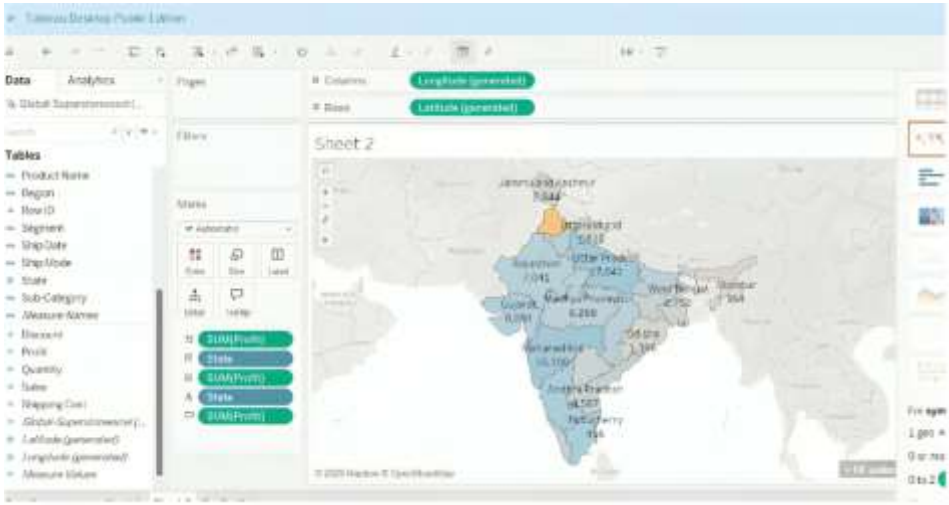
Step6:- Take one more sheet and save as Map chart,drag state from table section and drag on the sheet.



Step7:- drag profit from tables section and drop on color option of mark section to color the states as per their profit value.

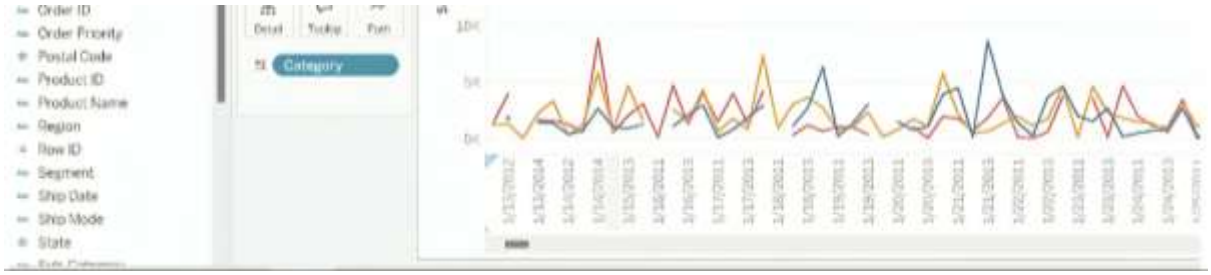


Step8:- If you want to see profit of state the drag profit in profit in Tooltip and to show name of state drag state in label also if you want to show profit of state the drag profit in label.

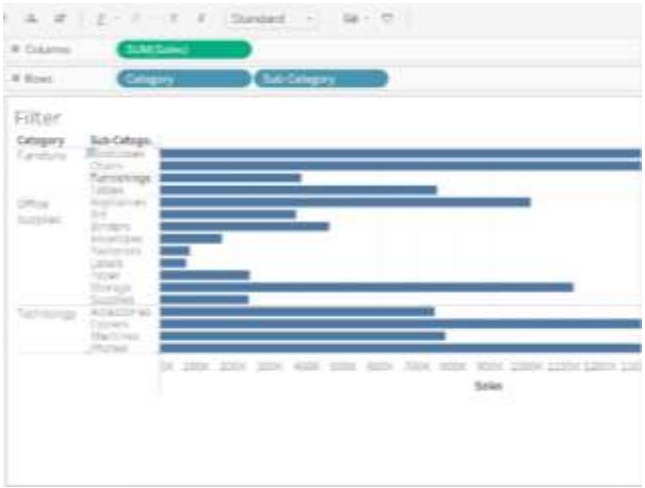


Step9:- Take one sheet and save it as Line chart,drag Order date from tables section and drop on column and click on add all members.

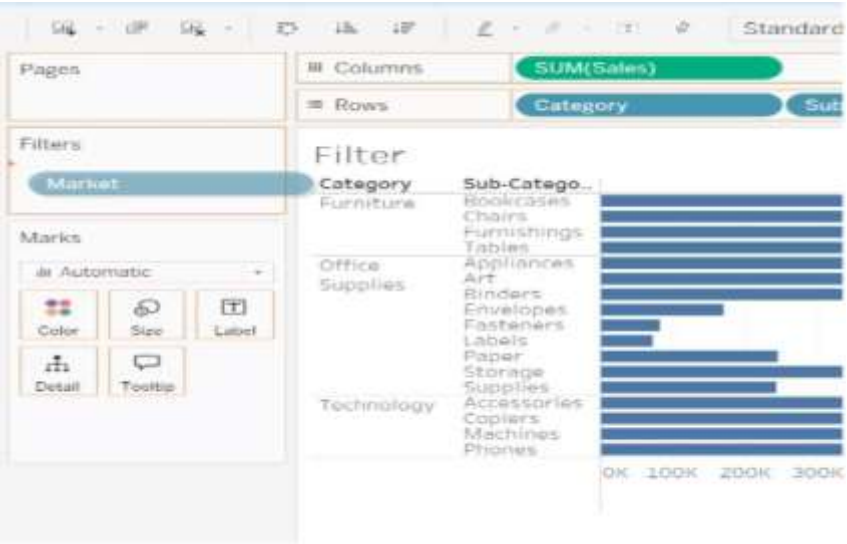
Step10:-Drag sales from tables section and drop on Rows.In Marks section select Line and drag Category from table section and drop on color in marks section.



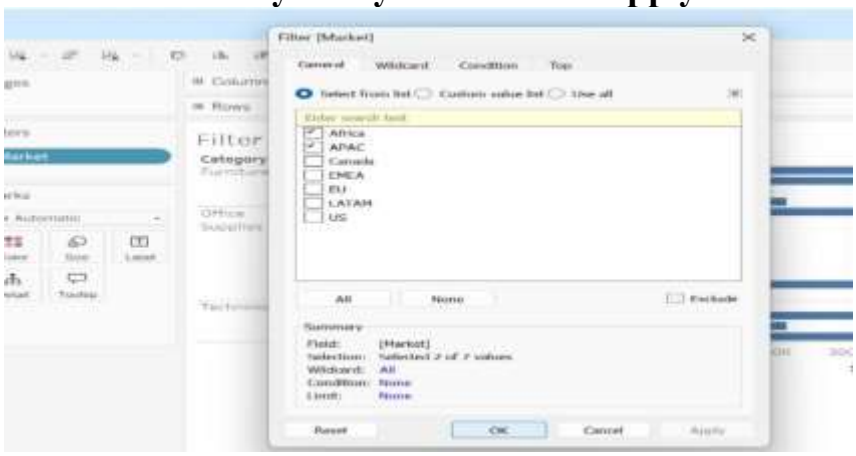
Step11:-Take one more sheet and name it as filter,drag sales from table section and drop on column and drag category from tables section and drop in Rows and Drag sub-category from tables section and drop on Rows,



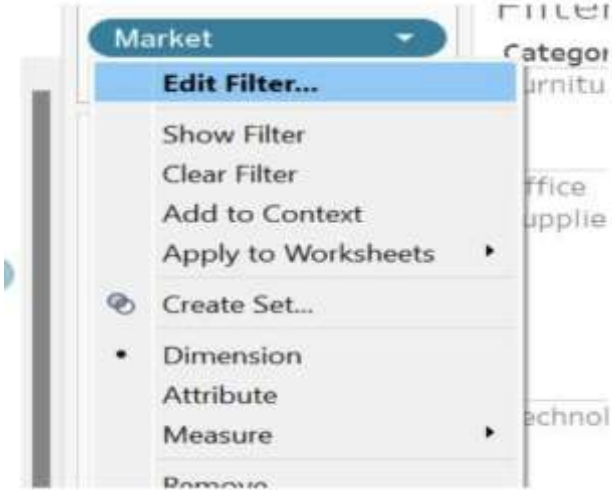
Step12:- If you want to filter based on market then drag Market from tables section and drop on filter.

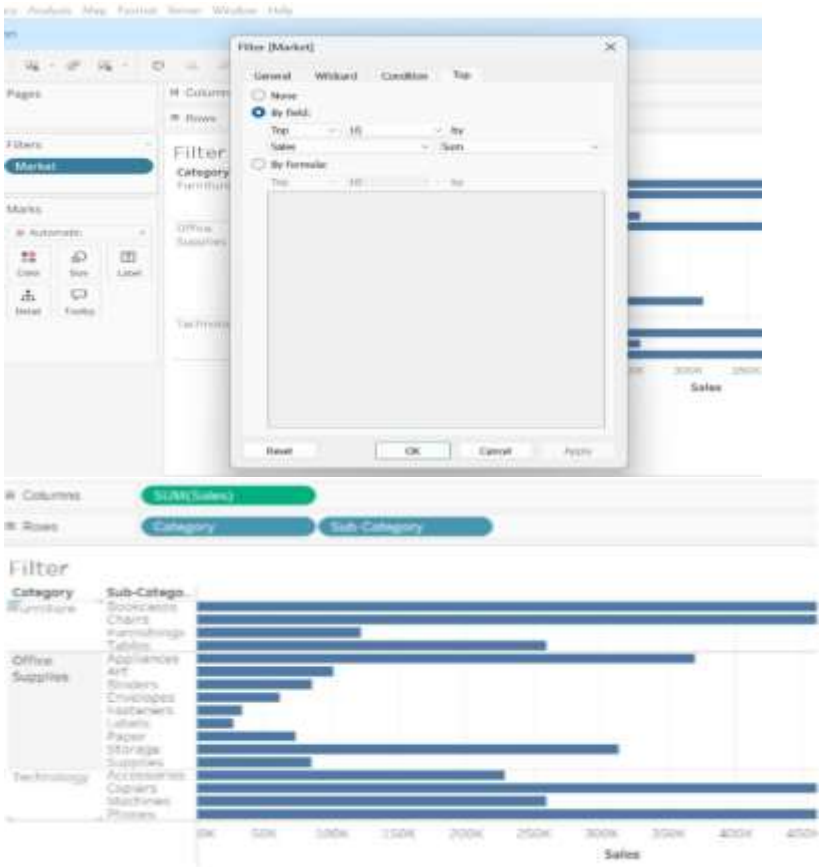


Select the country that you want then apply it and click OK.

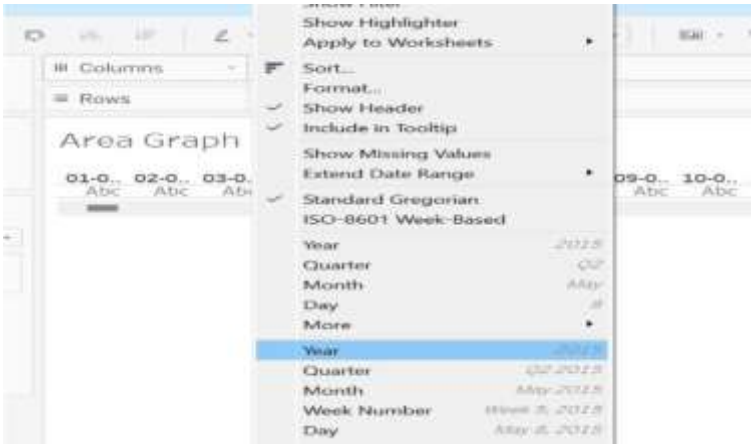


In market click on Edit filter then go to Top select By filed and write 10 and apply it and click on OK.

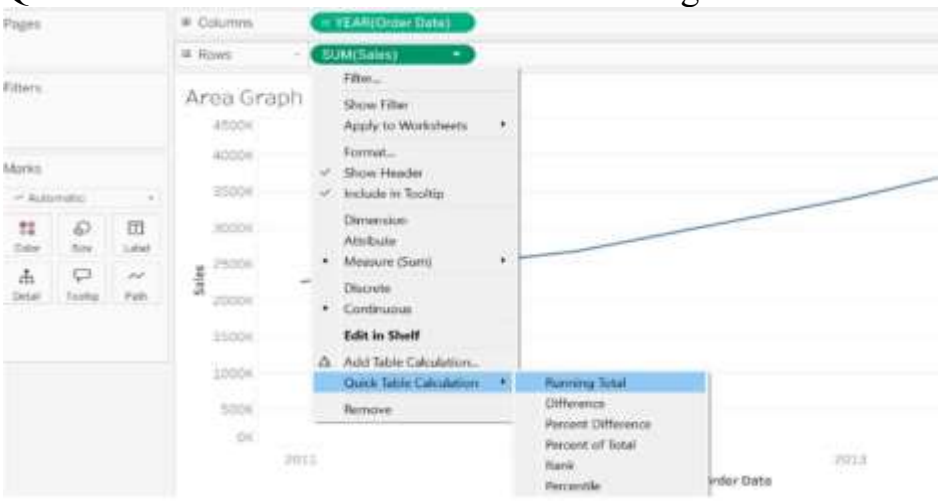




Step13:-Take a one more sheet and name it as Area Graph,drag order date from table section and drop in columns (make sure order date will be a date).

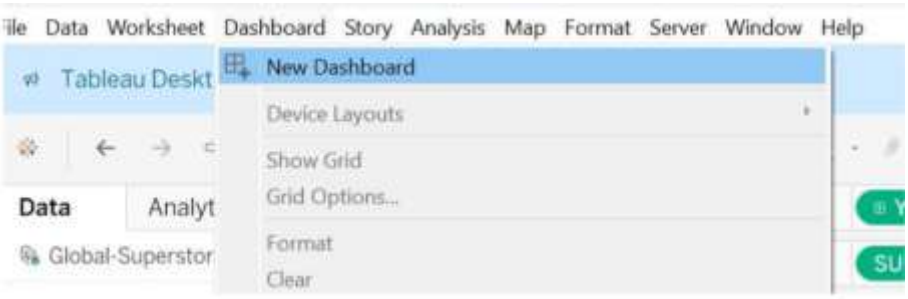


Step14:- Drag Sales from tables section and drop in Rows click on sales go to Quick Table Calculation and click on Running Total.

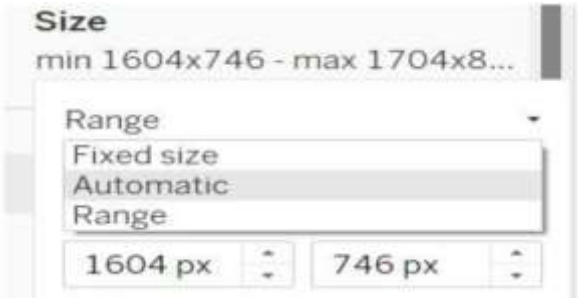


Step15:- Drag Category from table section and drop on color of market section, select Area in market section.

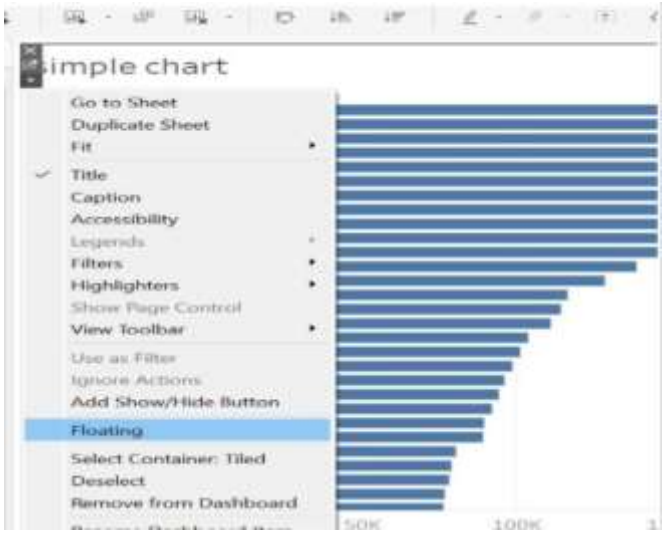
Step16:- create a dashboard



Click on size the go to Range and select Automatic.

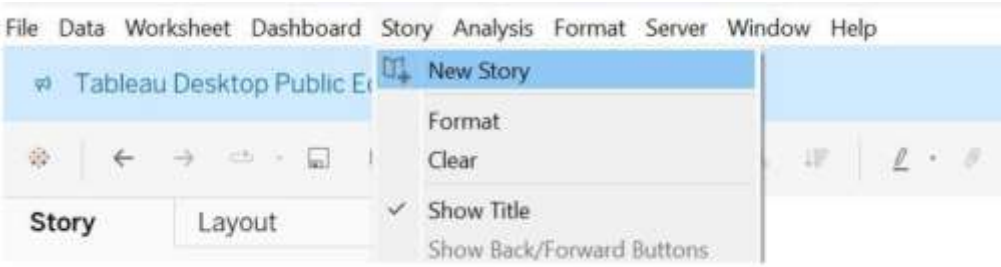


Drag sheet1 and drop on sheet then click on More option and select Floating.

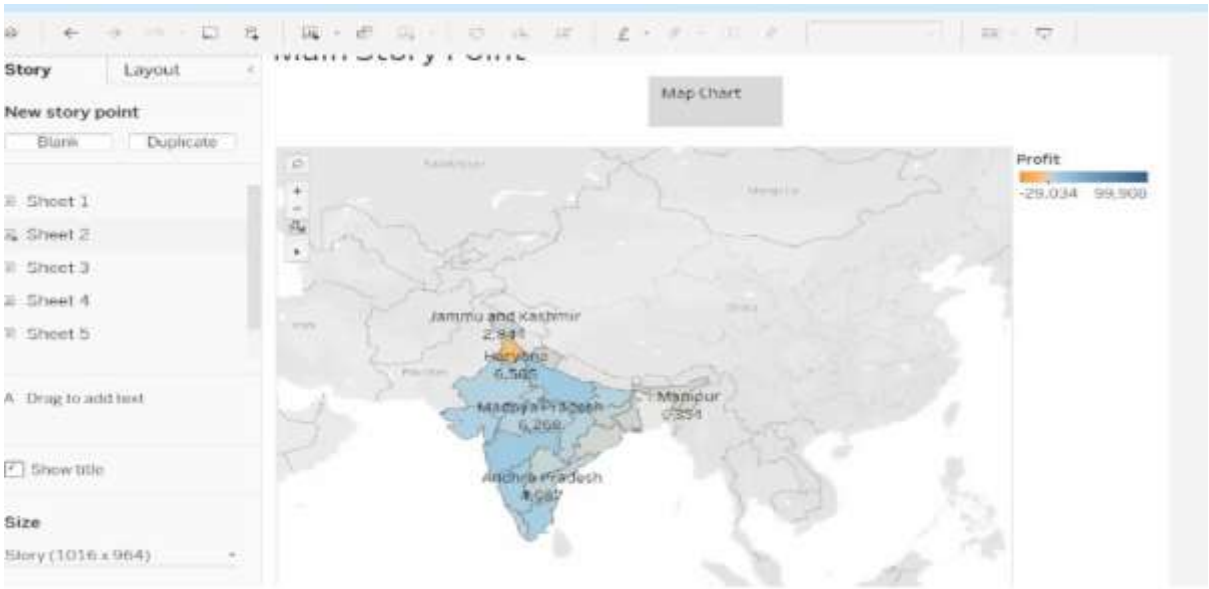


Do above step for all sheets and click S and 7.

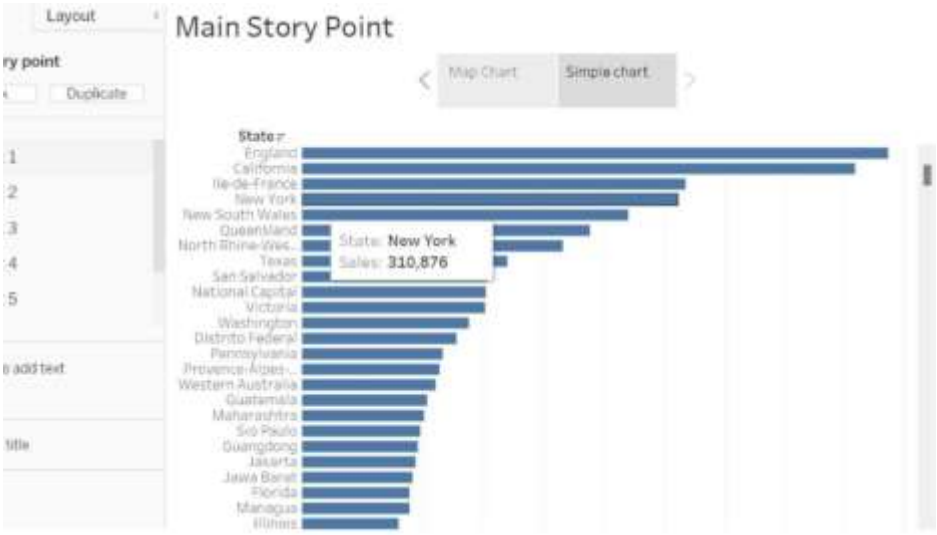
Step17:- create Story and give the as Main story point apply it and click on OK.



Add a caption as Map Chart drag a map on a sheet.



Drag next sheet and drop next to map chart and caption it simple chart.



Drag all the sheet and create a story.

Practical No.7

Aim : Regression and Its Types:
Program A: Single Variable
Input:

```
import pandas as pd
import seaborn as sns
from sklearn import linear_model
import warnings
df=pd.read_csv(r"D:\TYCS Shiv\Area_highprice.csv")
df
```

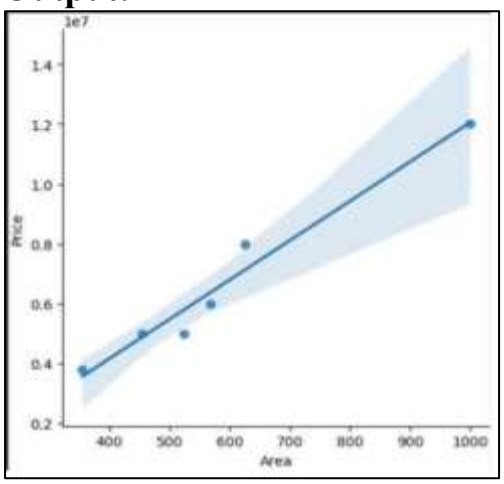
Output:

	Area	Price
0	567	6000000
1	355	3800000
2	455	5000000
3	626	8000000
4	1000	12000000
5	524	5000000

Input:

```
warnings.filterwarnings("ignore")
sns.lmplot(x="Area",y="Price",data=df)
```

Output:



Input:

```
reg=linear_model.LinearRegression()
reg.fit(df[["Area"]],df[["Price"]])
```

Output:

```
▾ LinearRegression
LinearRegression()
```

Input:

```
reg.predict([[500]])
```

Output:

```
array([[5477991.30275054]])
```

Input:

Reg.coef_

Output:

```
array([[13153.79920967]])
```

Input:

Reg.intercept_

Output:

```
array([-1098908.30208509])
```

Program B: Multi Variable

Input:

```
import pandas as pd
import seaborn as sns
from sklearn import linear_model
import warnings
df=pd.read_csv(r"D:\TYCS Shiv\Multiarea_pricing.csv")
df
```

Output:

	Bedrooms	Bathrooms	Sqft_living	Floors	Grade	Sqft_above	Sqft_basement	Price
0	3	1.00	1180	1.0	7	1180	0	221900
1	3	2.25	2570	2.0	7	2170	400	538000
2	2	1.00	770	1.0	6	770	0	180000
3	4	3.00	1960	NaN	7	1050	910	604000
4	3	2.00	1680	1.0	8	1680	0	510000
5	4	4.50	5420	1.0	11	3890	1530	267800
6	3	2.25	1715	2.0	7	1715	0	257500
7	3	1.50	1060	1.0	7	1060	0	291850
8	3	1.00	1780	1.0	7	1050	730	229500
9	3	2.50	1890	2.0	7	1890	0	323000
10	3	2.50	3560	1.0	8	1860	1700	662500

Input:

```
mean_f=df.Floors.mean()
mean_f1=round(mean_f)
mean_f1
```

Output: 1

Input:

df.floors

Output:

```
0     1.0
1     2.0
2     1.0
3     NaN
4     1.0
5     1.0
6     2.0
7     1.0
8     1.0
9     2.0
10    1.0
Name: Floors, dtype: float64
```

Input:

df.Floors=df.Floors.fillna(mean_f1)
df

Output:

	Bedrooms	Bathrooms	Sqft_living	Floors	Grade	Sqft_above	Sqft_basement	Price
0	3	1.00	1180	1.0	7	1180	0	221900
1	3	2.25	2570	2.0	7	2170	400	538000
2	2	1.00	770	1.0	6	770	0	180000
3	4	3.00	1960	1.0	7	1050	910	604000
4	3	2.00	1680	1.0	8	1680	0	510000
5	4	4.50	5420	1.0	11	3890	1530	267800
6	3	2.25	1715	2.0	7	1715	0	257500
7	3	1.50	1060	1.0	7	1060	0	291850
8	3	1.00	1780	1.0	7	1050	730	229500
9	3	2.50	1890	2.0	7	1890	0	323000
10	3	2.50	3560	1.0	8	1860	1700	662500

Input:

reg=linear_model.LinearRegression()
reg.fit(df[["Bedrooms","Bathrooms","Sqft_living","Floors","Grade","Sqft_above","Sqft_basement"]],df["Price"])

Output:

```
▾ LinearRegression
LinearRegression()
```

Input:

reg.predict([[3,2,1180,1,7,1180,0]])

Output:

```
array([337141.76818324])
```

Program C: Summary of the Model

Input:

import pandas as pd data=pd.read_csv(r"D:\TYCS Shiv\homeprices.csv")
print(data.head())

Output:

	area	bedrooms	age	price
0	2600	3	20	550000
1	3000	4	15	565000
2	3200	4	18	610000
3	3600	3	30	595000
4	4000	5	8	760000

Input:

import statsmodels.api as sm
x=data[["area","bedrooms","age"]]
y=data["price"]
X=sm.add_constant(x)
model=sm.OLS(y,X).fit()
print(model.summary())

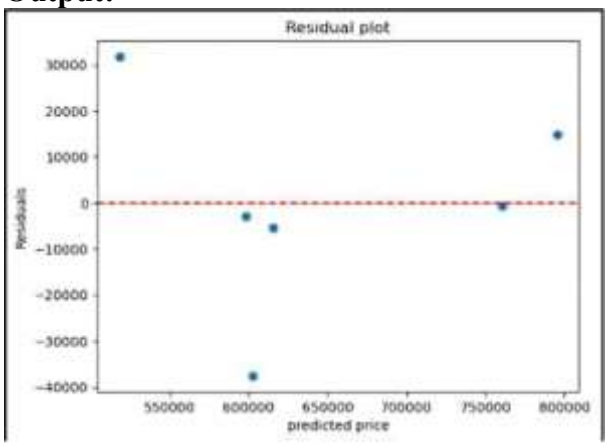
Output:

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  price   R-squared:                       0.955
Model:                            OLS   Adj. R-squared:                  0.888
Method:                 Least Squares   F-statistic:                     14.15
Date:                Wed, 21 Jan 2026   Prob (F-statistic):             0.0667
Time:                        08:41:15   Log-Likelihood:                -68.263
No. Observations:                   6   AIC:                             144.5
Df Residuals:                       2   BIC:                             143.7
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const       2.213e+05   1.92e+05      1.150      0.369   -6.07e+05    1.05e+06
area         112.0624     57.291      1.956      0.190    -134.442     358.567
bedrooms    2.339e+04   5.37e+04      0.435      0.706   -2.08e+05    2.55e+05
age        -3231.7179   5582.527     -0.579      0.621   -2.73e+04    2.08e+04
==============================================================================
Omnibus:                          nan   Durbin-Watson:                   2.279
Prob(Omnibus):                    nan   Jarque-Bera (JB):                0.148
Skew:                          -0.318   Prob(JB):                        0.928
Kurtosis:                       2.565   Cond. No.                     4.56e+04
==============================================================================
```

Input:

```
import matplotlib.pyplot as plt
residuals=y-model.predict(X)
plt.scatter(model.predict(X),residuals) plt.axhline(y=0,color="r",linestyle="--")
plt.xlabel("predicted price")
plt.ylabel("Residuals")
plt.title("Residual plot")
plt.show()
```

Output:



Practical No.8

Aim :- Logistic Regression and Decision Tree

1. **Build a logistic regression model to predict a binary outcome.**

   Input:

   ```
   from sklearn.linear model import LogisticRegression
   from sklearn.metrics import accuracy _ score, precision score, recall _ score,
   ```

fl score,confusion matrix,
classification _ report
import numpy as np
y = np.array([0, 0, 0, l, l])
model LogisticRegression()
model.fit(X, y)

Output:

```
▾ LogisticRegression
LogisticRegression()
```

Input:

n_data = np.array([[4.5]])
y_pred = model.predict(n_data)
print("Prediction: ", y_pred)

Output:

Prediction: [1]

Input:

prob = model.predict_proba(n_data)
print("Predict Probabilities:", prob)

Output:

Predict Probabilities: [[0.00120307 0.99879693]]

2. **Evaluate the model's performance using classification metrics (e.g., accuracy, precision, recall).**

Input:

accuracy = accuracy_score(y, model.predict(X))
print(f"Accuracy: {accuracy:.2f}")
precision = precision_score(y, model.predict(X))
print(f"Precision: {precision:.2f}")
f1 = f1_score(y, model.predict(X), average="binary")
print(f"F1 Score: {f1:.2f}")

Output:

```
Accuracy: 1.00
Precision: 1.00
F1 Score: 1.00
```

Input:

conf_matrix = confusion_matrix(y, model.predict(X))
print("Confusion Matrix:\n", conf_matrix)

Output:

```
Confusion Matrix:
 [[3 0]
 [0 2]]
```

Input:

print("\nClassification Report:")
print(classification_report(y, model.predict(X)))

Output:

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         3
           1       1.00      1.00      1.00         2

    accuracy                           1.00         5
   macro avg       1.00      1.00      1.00         5
weighted avg       1.00      1.00      1.00         5
```

3. **Construct a decision tree model and interpret the decision rules for classification.**

Input:

```
from sklearn.tree import DecisionTreeClassifier, export_text
import numpy as np
x = np.array([[1, 50], [2, 60], [3, 65], [4, 75], [5, 90]])
y = np.array([0, 0, 0, 1, 1])
model = DecisionTreeClassifier()
model.fit(x, y)
```

Output:

```
▾ DecisionTreeClassifier
DecisionTreeClassifier()
```

Input:

```
tree_rules export_text(model,
feature_names=["Years_Experience","Performance_Score"])
print("Decision Tree Rules:\n")
print(tree_rules)
```

Output:

```
Decision Tree Rules:

|--- Performance_Score <= 70.00
|   |--- class: 0
|--- Performance_Score >  70.00
|   |--- class: 1
```

Input:

```
new_data = np.array([[4, 72]])
prediction = model.predict(new_data)
print("\nPrediction for [4 Years, 72 Score]:","Promoted" if prediction[0] == 1 else "Not
Promoted")
```

Output:

Prediction for [4 Years, 72 Score]: Promoted

Practical No.9

Aim: Clustering With K Means

Input:

```
from sklearn.cluster import KMeans
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from matplotlib import pyplot as plt
%matplotlib inline
from google.colab import files

uploaded = files.upload()
df = pd.read_csv("income.csv")
df.head()
```

Output :

Choose Files income.csv

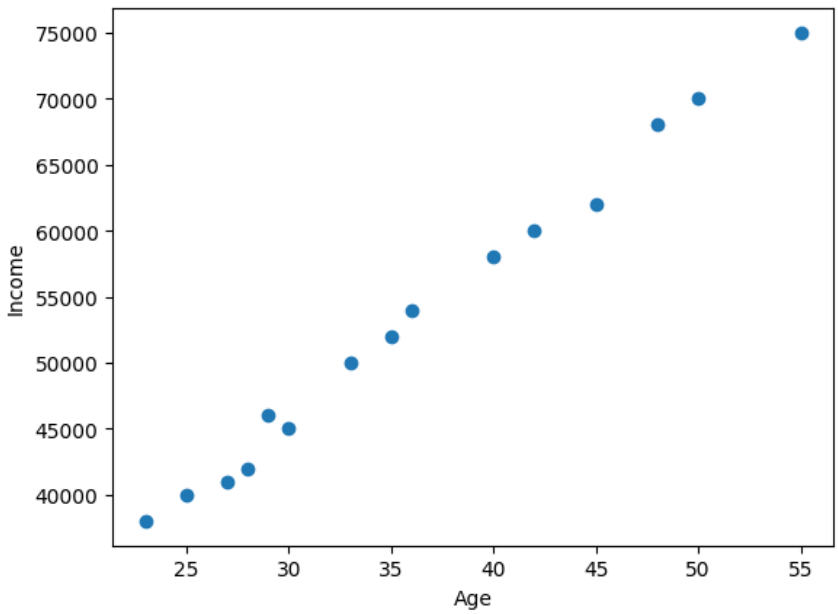
income.csv(text/csv) - 419 bytes, last modified: 2/5/2026 - 100% done

Saving income.csv to income (1).csv

	ID	Age	Income	Education	Gender
0	1	25	40000	Bachelors	Male
1	2	30	45000	Masters	Female
2	3	28	42000	Bachelors	Female
3	4	35	52000	Masters	Male
4	5	40	58000	PhD	Male

```
plt.scatter(df.Age,df["Income"])
plt.xlabel("Age")
plt.ylabel("Income")
```

```
Text(0, 0.5, 'Income')
```



```
km = KMeans(n_clusters=3)
y_predicted = km.fit_predict(df[["Age","Income"]])
y_predicted
```

```
array([2, 2, 2, 0, 0, 0, 2, 1, 1, 0, 2, 0, 0, 2, 1], dtype=int32)
```

```
df["cluster"]=y_predicted
df.head()
```

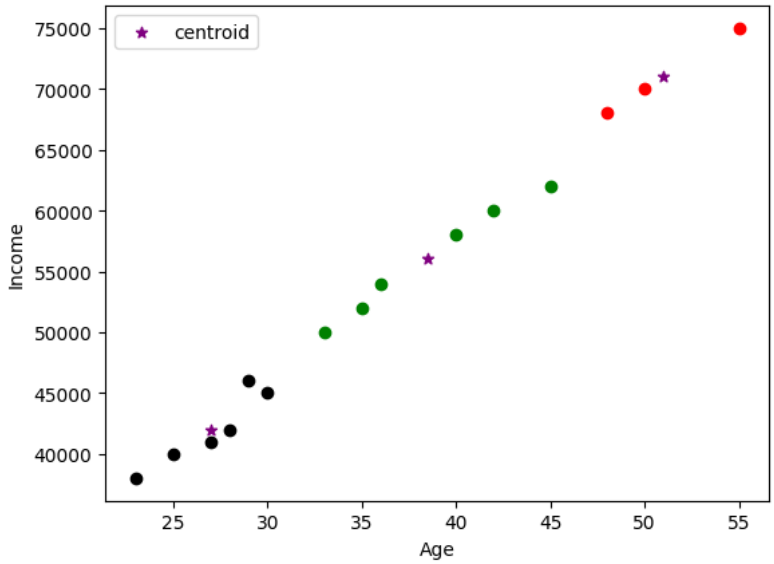
	ID	Age	Income	Education	Gender	cluster
0	1	25	40000	Bachelors	Male	2
1	2	30	45000	Masters	Female	2
2	3	28	42000	Bachelors	Female	2
3	4	35	52000	Masters	Male	0
4	5	40	58000	PhD	Male	0

```
km.cluster_centers_
```

```
array([[3.85e+01, 5.60e+04],
       [5.10e+01, 7.10e+04],
       [2.70e+01, 4.20e+04]])
```

```
df1 = df[df.cluster==0]
df2 = df[df.cluster==1]
df3 = df[df.cluster==2]
plt.scatter(df1.Age,df1["Income"],color="green")
plt.scatter(df2.Age,df2["Income"],color="red")
plt.scatter(df3.Age,df3["Income"],color="black")
plt.scatter(km.cluster_centers_[:,0],km.cluster_centers_[:,1],color="purple",mar
ker="*",label="centroid")
plt.xlabel("Age")
plt.ylabel("Income")
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7a50f40813a0>
```



❖ Preprocessing using min max scaler

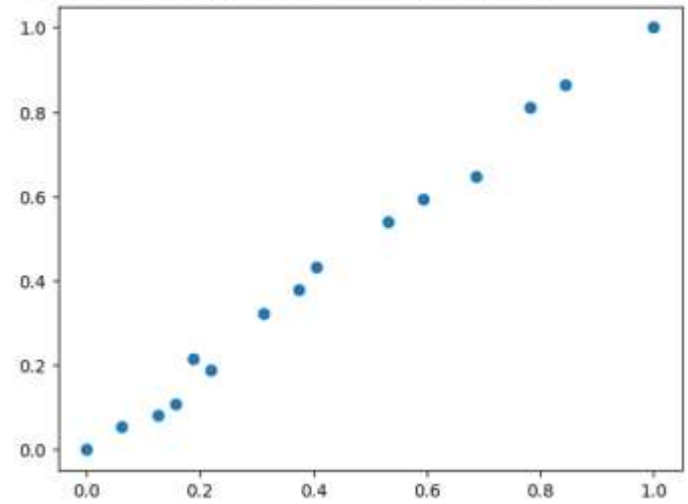
Input :

```
scaler = MinMaxScaler()
scaler.fit(df[["Income"]])
df["Income"] = scaler.transform(df[["Income"]])
scaler.fit(df[["Age"]])
df["Age"] = scaler.transform(df[["Age"]])
df.head()
```

	ID	Age	Income	Education	Gender	cluster
0	1	0.06250	0.054054	Bachelors	Male	2
1	2	0.21875	0.189189	Masters	Female	2
2	3	0.15625	0.108108	Bachelors	Female	2
3	4	0.37500	0.378378	Masters	Male	0
4	5	0.53125	0.540541	PhD	Male	0

```
plt.scatter(df.Age,df["Income"])
```

```
<matplotlib.collections.PathCollection at 0x7a50f4249cd0>
```



km = KMeans(n_clusters=3)
y_predicted = km.fit_predict(df[["Age","Income"]])
y_predicted

```
array([2, 2, 2, 0, 0, 1, 2, 1, 1, 0, 2, 0, 0, 2, 1], dtype=int32)
```

df["cluster"]=y_predicted
df.head()

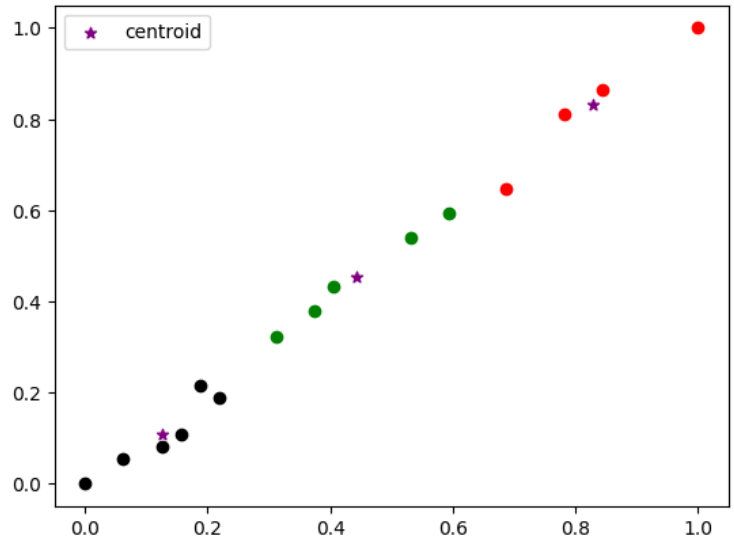
	ID	Age	Income	Education	Gender	cluster
0	1	0.06250	0.054054	Bachelors	Male	2
1	2	0.21875	0.189189	Masters	Female	2
2	3	0.15625	0.108108	Bachelors	Female	2
3	4	0.37500	0.378378	Masters	Male	0
4	5	0.53125	0.540541	PhD	Male	0

km.cluster_centers_

```
array([[0.44375   , 0.45405405],
       [0.828125  , 0.83108108],
       [0.125     , 0.10810811]])
```

df1 = df[df.cluster==0]
df2 = df[df.cluster==1]
df3 = df[df.cluster==2]
plt.scatter(df1.Age,df1["Income"],color="green")
plt.scatter(df2.Age,df2["Income"],color="red")
plt.scatter(df3.Age,df3["Income"],color="black")
plt.scatter(km.cluster_centers_[:,0],km.cluster_centers_[:,1],color="purple",marker="*",label="centroid")
plt.legend()

```
<matplotlib.legend.Legend at 0x7a50f5381f40>
```

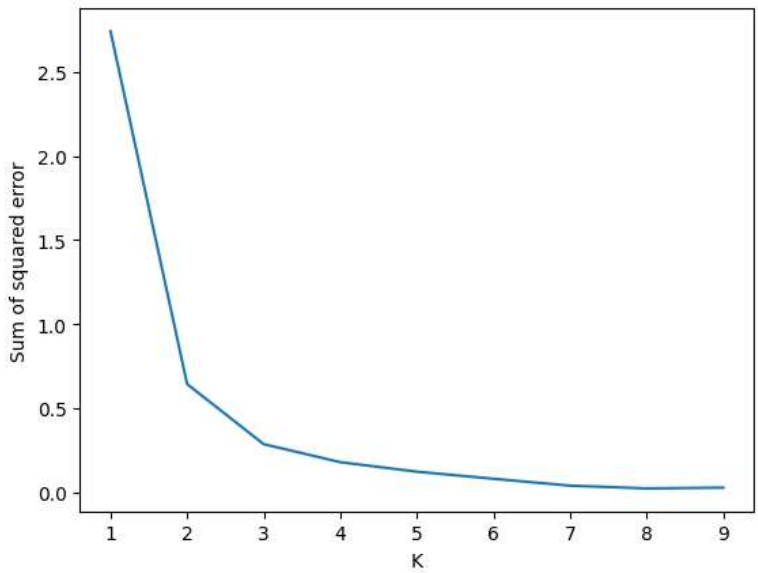


❖ Elbow Plot

```
sse = []
k_rng = range(1,10)
for k in k_rng:
  km = KMeans(n_clusters=k)
  km.fit(df[["Age","Income"]])
  sse.append(km.inertia_)

plt.xlabel("K")
plt.ylabel("Sum of squared error")
plt.plot(k_rng,sse)
```

Output:

```
[<matplotlib.lines.Line2D at 0x7a50f000ccb0>]
```



Practical No.10

Aim: Perform the data clustering using clustering algorithm.

k-means clustering using R

```
#apply K means to iris and store result
newiris <- iris
newiris$Species <- NULL
(kc <- kmeans(newiris,3))
```

Output:

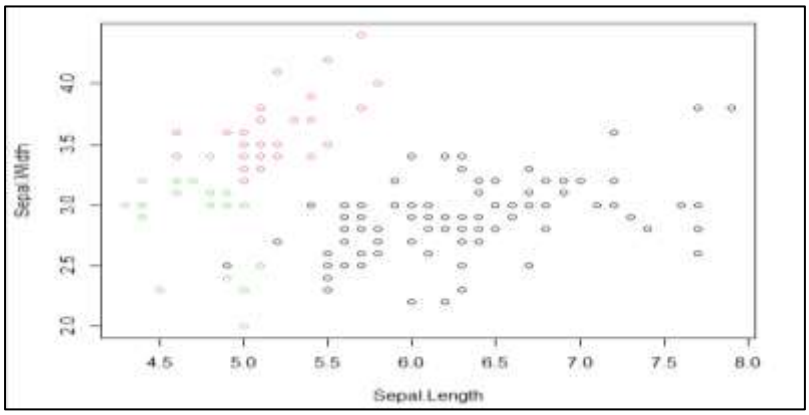
```
K-means clustering with 3 clusters of sizes 96, 33, 21

Cluster means:
  Sepal.Length Sepal.Width Petal.Length Petal.Width
1     6.314583    2.895833     4.973958   1.7031250
2     5.175758    3.624242     1.472727   0.2727273
3     4.738095    2.904762     1.790476   0.3523810

Clustering vector:
  [1] 2 3 3 3 2 2 2 2 3 3 2 2 3 3 2 2 2 2 2 2 2 2 2 2 3 3 2 2 2 3 3 2 2 2 3 2 2 2 3 2 2 3 3 2 2 3 2 3 2 2 1 1 1 1 1 1 1 3 1 1 3 1 1 1 1 1 1 1 1 1
 [71] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 3 1 1 1 1 3 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
[141] 1 1 1 1 1 1 1 1 1 1

Within cluster sum of squares by cluster:
[1] 118.651875   6.432121  17.669524
 (between_SS / total_SS =  79.0 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss" "betweenss"    "size"         "iter"         "ifault"
```

```
table(iris$Species,kc$cluster)
```

Output:

```
             1  2  3
  setosa     0 33 17
  versicolor 46  0  4
  virginica  50  0  0
```

Input:

```
#Plot the clusters and their centers
plot(newiris[c("Sepal.Length","Sepal.Width")],col=kc$cluster)
points(kc$centers[,c("Sepal.Length","Sepal.Width")],col=1:3,pch=8,cex=2)
dev.off()
#Plot the clusters and their centre
plot(newiris[c("Sepal.Length", "Sepal.Width")],col=kc$cluster)
```

Output: